

Sumários disponíveis no [ScienceDirect](#)

Computadores e Educação

Página inicial da revista: www.elsevier.com/locate/compedu

Transição de blocos introdutórios para textos. ambientes para linguagens de programação profissionais em alto nível salas de aula de informática escolar

David Weintropa,^y Uri Wilenskyb^y Faculdade de Educação, Faculdade de Estudos da Informação, Universidade de Maryland, College Park, EUA^b Centro de Aprendizagem Conectada e Modelagem Computacional, Universidade Northwestern, EUA

INFORMAÇÕES DO ARTIGO

Palavras-chave:

Avaliação de sistemas CAL
Ambientes de aprendizagem interativos
Programação e linguagens de programação
Ensino secundário
Estratégias de ensino/aprendizagem

RESUMO

Linguagens de programação baseadas em blocos estão se tornando cada vez mais comuns em cursos introdutórios de ciência da computação em toda a educação básica. Uma justificativa para o uso de linguagens baseadas em blocos é que elas são frequentemente utilizadas em cursos de informática introdutórios. ambientes em contextos educacionais formais é a ideia de que os conceitos e práticas desenvolvidos O uso dessas ferramentas introdutórias preparará os alunos para futuras oportunidades de aprendizado em ciência da computação. Essa visão se baseia na premissa de que o aprendizado atitudinal e conceitual Os ganhos obtidos durante o trabalho em ambientes introdutórios baseados em blocos serão transferidos para linguagens de programação textuais convencionais. Para testar essa hipótese, este artigo apresenta o Resultados de um estudo quase-experimental em sala de aula, no qual iniciantes em programação passaram cinco semanas utilizando um ambiente de programação baseado em blocos ou em texto. Após cinco semanas com a ferramenta introdutória, os alunos passaram a usar Java, uma linguagem de programação convencional baseada em texto. O estudo acompanhou os alunos durante 10 semanas após a transição. Ao longo das 15 semanas Foram aplicadas avaliações de estudo, atitudinais e conceituais, além de programas elaborados por estudantes. Foram coletados dados sobre aprendizagem conceitual, mudanças de atitude e alterações nas práticas de programação. foram analisadas para avaliar como a modalidade introdutória impactou os alunos durante a transição para uma linguagem de programação profissional baseada em texto. As conclusões deste estudo complementam pesquisas anteriores. trabalho que constatou uma diferença no desempenho em avaliações de conteúdo após a introdução parte do estudo (Weintrop & Wilensky, 2017a). Este artigo mostra que a diferença na aprendizagem conceitual que surgiu após cinco semanas entre as condições baseadas em blocos e baseadas em texto desaparece após 10 semanas em Java. Nenhuma diferença nas práticas de programação foi encontrada entre as duas condições ao trabalhar em Java. Da mesma forma, diferenças nas medidas atitudinais que Os efeitos observados após o trabalho em ambientes introdutórios também desapareceram após 10 semanas em Java, resultando em nenhuma diferença entre as condições após 15 semanas. A contribuição deste trabalho é... Este artigo apresenta o primeiro estudo quase-experimental da transferência de conhecimento entre ambientes baseados em blocos e em texto em um contexto de aprendizado de ciência da computação. O objetivo deste trabalho é aprofundar nossa compreensão dos benefícios e limitações das ferramentas de programação em blocos na preparação de alunos para o futuro aprendizado da ciência da computação. Contexto do ensino médio. Discute-se a ausência de diferenças significativas entre as duas modalidades introdutórias de programação após a transição dos alunos para linguagens de programação profissionais. juntamente com as implicações dessas descobertas para pesquisadores da área de educação em ciência da computação e educadores, bem como a comunidade mais ampla de pesquisadores que estudam o papel da tecnologia em educação.

^y Autor correspondente.Endereço de e-mail: weintrop@umd.edu (D. Weintrop).<https://doi.org/10.1016/j.compedu.2019.103646>

Recebido em 1 de agosto de 2018; Recebido em versão revisada em 30 de julho de 2019; Aceito em 4 de agosto de 2019.

Disponível online em 07 de agosto de 2019.

0360-1315/ © 2019 Elsevier Ltd. Todos os direitos reservados.

1. Introdução

A programação em blocos está se tornando rapidamente a forma como os alunos mais jovens são introduzidos ao campo da ciência da computação (Bau, Gray, Kelleher, Sheldon e Turbak, 2017). Impulsionados pela popularidade de ferramentas como Scratch (Resnick et al., 2009), Blockly (Fraser, 2015) e Alice (Cooper, Dann e Pausch, 2000), milhões de crianças estão interagindo com a programação por meio de ferramentas gráficas de arrastar e soltar. Por exemplo, a iniciativa Hora do Código da Code.org, que inclui dezenas de atividades que incorporam programação em blocos, ultrapassou recentemente a marca de 500 milhões de "horas dedicadas" e alcançou alunos em todos os países do planeta (Code.org, 2017). Isso destaca como o entusiasmo pela ciência da computação é um fenômeno global. Devido, em parte, ao sucesso dessas ferramentas e iniciativas em engajar iniciantes na programação, os ambientes de programação em blocos estão sendo cada vez mais incorporados aos currículos de ciência da computação do ensino médio. Exemplos desses currículos incluem Exploring Computer Science (Goode, Chapman e Margolis, 2012), o projeto Computer Science Principles (Cuny, 2015) e as ofertas curriculares da Code.org (Code.org Curricula, 2019).

Muitos usos de ferramentas baseadas em blocos em contextos educacionais formais pressupõem que tais ferramentas ajudarão a preparar os alunos para o ensino posterior de linguagens textuais (Armoni, Meerbaum-Salant e Ben-Ari, 2015; Brown et al., 2016; Dann, Cosgrove, Slater, Culyba e Cooper, 2012). Essa transição geralmente faz parte da trajetória mais ampla da ciência da computação, na qual cursos introdutórios baseados em blocos visam preparar os alunos para a transição para linguagens textuais profissionais. Isso pode ser observado na forma como a transição tem sido estudada até o momento (por exemplo, Armoni et al., 2015; Dann et al., 2012; Powers, Ecott e Hirshfield, 2007). Essa suposição também foi corroborada pelos alunos do ensino médio que participaram do estudo apresentado neste trabalho, que fizeram declarações como "[a programação em blocos] é um bom começo; depois que aprendermos os comandos e tudo mais, podemos passar para Java" e "[a programação em blocos] está nos preparando para o que faremos no futuro". Embora já existam trabalhos focados na aprendizagem que ocorre com o uso de ferramentas baseadas em blocos (por exemplo, Franklin et al., 2017; Grover & Basu, 2017; Weintrop & Wilensky, 2015a), poucos testaram rigorosamente a transição de ferramentas introdutórias baseadas em blocos para linguagens textuais em contextos formais (Blikstein, 2018; Shapiro & Ahrens, 2016). Uma revisão sistemática da literatura sobre o papel da programação visual concluiu que há incertezas quanto à eficácia das linguagens baseadas em blocos quando se olha além de contextos introdutórios (Noone & Mooney, 2018). Essa questão é de grande importância, dado o papel crescente das ferramentas baseadas em blocos na educação básica em todo o mundo e seu impacto no ensino e na aprendizagem da ciência da computação (Blikstein, 2018; Caspersen, 2018).

Este artigo busca compreender se e como a modalidade utilizada (baseada em blocos versus baseada em texto) prepara os alunos para línguas textuais convencionais. Essa linha de investigação é relevante tanto para a comunidade de pesquisa, por se tratar de uma questão em aberto que pode orientar futuras pesquisas sobre design e aprendizagem, quanto para os profissionais que, diariamente, precisam tomar decisões sobre ambientes de aprendizagem e pedagogia em suas salas de aula. Mais especificamente, este artigo responde à seguinte questão de pesquisa:

Nas aulas introdutórias de ciência da computação do ensino médio, como a modalidade utilizada para o ensino de programação introdutória influencia o aprendizado? Qual o impacto da programação baseada em blocos versus a programação baseada em texto nos alunos quando fazem a transição para uma linguagem de programação profissional baseada em texto?

Este artigo apresenta os resultados de um estudo quase-experimental concebido para responder a essa questão. O estudo foi realizado em duas turmas de ciência da computação do ensino médio e compara ambientes de programação isomórficos, baseados em blocos e em texto. Os alunos passaram cinco semanas trabalhando em uma versão baseada em blocos ou em texto do mesmo ambiente de programação introdutório antes de migrarem para Java. O mesmo professor ministrou as duas turmas e os alunos em cada condição utilizaram o mesmo currículo e tiveram o mesmo tempo de dedicação às tarefas. Os resultados das primeiras cinco semanas do estudo são relatados em (Weintrop & Wilensky, 2017a). Este trabalho dá continuidade a esse artigo, focando especificamente no que aconteceu após a transição dos ambientes introdutórios para Java.

Para entender como o design de ferramentas introdutórias prepara os alunos para a programação em linguagens de programação profissionais, apresentamos resultados comparativos de avaliações de conteúdo, pesquisas de opinião e investigamos práticas de programação que surgiram após a transição para Java.

2. Trabalhos anteriores

2.1. Programação baseada em blocos

A programação baseada em blocos é um paradigma de programação visual que utiliza uma metáfora de primitivas de programação como peças de um quebra-cabeça para tornar o ato de programar mais acessível e intuitivo para iniciantes (Bau et al., 2017; Good, 2018). A abordagem de programação baseada em blocos está se tornando cada vez mais difundida. Duncan, Bell e Tanimoto (2014) revisaram 47 ambientes introdutórios de programação e descobriram que 28 deles utilizavam a abordagem de programação baseada em blocos, incluindo 19 dos 24 ambientes projetados para alunos com menos de 8 anos de idade. Escrever um programa em um ambiente de programação baseado em blocos consiste em encaixar comandos arrastando e soltando-os lado a lado. Os ambientes de programação baseados em blocos incluem diversos recursos projetados para facilitar o ato de programar (Maloney, Resnick, Rusk, Silverman e Eastmond, 2010; Tempel, 2013). Por exemplo, a representação visual de um bloco fornece pistas que indicam como e onde um determinado comando pode ser usado. Essas pistas visuais também são usadas como meio de reforçar regras sintáticas, impedindo que instruções incompatíveis ou incorretas sejam combinadas para criar instruções inválidas. Dessa forma, a modalidade baseada em blocos previne erros de sintaxe durante a construção do programa, mas mantém a prática de escrever programas instrução por instrução. Os ambientes de programação baseados em blocos também auxiliam o programador apresentando os comandos disponíveis em gavetas organizadas logicamente e de fácil navegação, um recurso identificado por aprendizes como facilitador da aprendizagem da programação (Weintrop & Wilensky, 2015b). Em conjunto, esses recursos fornecem uma rica rede de suporte para iniciantes, tornando-os uma maneira atraente de introduzi-los à programação (Kölling & McKay, 2016; Weintrop & Wilensky, 2017b).

Pesquisas que analisam o uso de ambientes de programação baseados em blocos em contextos educacionais introdutórios do ensino fundamental e médio estão mostrando que...

Pode ser uma forma eficaz de apresentar conceitos básicos de programação a iniciantes. No ensino fundamental II (10 a 14 anos), ferramentas baseadas em blocos são uma maneira eficaz de tornar os conceitos de programação acessíveis a alunos mais jovens, quando oferecidas em um currículo apropriado ao seu desenvolvimento (Franklin et al., 2017; Hill, Dwyer, Martinez, Harlow e Franklin, 2015; Howland e Good, 2014; Meerbaum-Salant, Armoni e Ben-Ari, 2010). Após ensinar um currículo de ciência da computação para alunos do ensino fundamental II, Grover et al. (2015) descobriram que a programação baseada em blocos é uma maneira produtiva de apresentar aos alunos conceitos básicos de computação e desenvolver importantes habilidades de pensamento computacional, como o pensamento algorítmico. Estudos comparativos entre ambientes baseados em blocos e em texto, com alunos nessa faixa etária, constataram que os aprendizes têm desempenho comparável ou superior no ambiente baseado em blocos e concluem as tarefas mais rapidamente (Lewis, 2010; Matsuzawa et al., 2015; Price & Barnes, 2015, pp. 91–99). No ensino médio, um estudo comparativo revelou que os alunos apresentam melhor desempenho em avaliações conceituais após trabalharem com ferramentas baseadas em blocos, em comparação com alternativas isomórficas baseadas em texto, após cinco semanas de instrução (Weintrop & Wilensky, 2017a). Em conjunto, esses trabalhos corroboram a decisão de utilizar a modalidade de programação baseada em blocos nas salas de aula do ensino fundamental e médio.

Além dos ganhos conceituais, ambientes de programação baseados em blocos têm demonstrado impactos positivos na motivação, nas atitudes e no engajamento de alunos do ensino fundamental e médio. Isso foi constatado em diversos ambientes de programação, incluindo Scratch (Malan & Leitner, 2007, pp. 223–227; Resnick et al., 2009; Ruf, Mühling, & Hubwieser, 2014, pp. 50–59), Snap! (Garcia, Harvey e Barnes, 2015) e Alice (Kelleher, Pausch e Kiesler, 2007; Kelleher e Pausch, 2007). É importante notar também que a abordagem de programação baseada em blocos tem sido bem-sucedida em envolver programadores iniciantes de populações historicamente sub-representadas (Kelleher et al., 2007; Maloney, Peppler, Kafai, Resnick e Rusk, 2008; Tangney, Oldham, Conneely, Barrett e Lawlor, 2010; Wilson e Moffat, 2010).

2.2. Ambientes de programação de modalidade dupla e bidirecional

Uma área ativa de pesquisa relevante para o presente estudo é o trabalho de design que explora maneiras de combinar a programação em blocos com recursos de representações textuais convencionais de ideias computacionais. A ideia por trás dessa abordagem é utilizar múltiplas modalidades para auxiliar os alunos a compreenderem os conceitos de programação e a criarem programas bem-sucedidos (Good & Howland, 2017; Weintrop & Holbert, 2017). Isso faz parte da discussão mais ampla sobre o papel a longo prazo da programação em blocos na educação e em outras áreas (Shapiro & Ahrens, 2016; Weintrop, 2019). Dividimos esse conjunto de pesquisas em duas categorias: ambientes de dupla modalidade, que combinam recursos de ferramentas baseadas em blocos e em texto em uma única interface, e ambientes bidirecionais que permitem aos alunos alternar entre apresentações do programa baseadas em blocos e em texto. Esses ambientes são pertinentes a este estudo, pois representam alternativas à noção de que a programação em blocos serve como precursora da programação baseada em texto.

Há um número crescente de ambientes de programação bidirecional que estão ganhando popularidade nos círculos de ensino de ciência da computação. O editor Droplet (Bau, 2015), usado no AppLap do Code.org e no ambiente Pencil Code, oferece um botão que permite ao aluno escolher entre trabalhar em uma interface baseada em blocos ou em texto. Clicar no botão de transição inicia uma animação na qual o aluno pode observar seu código se transformar de uma modalidade para a outra, reforçando a equivalência entre as duas apresentações do código. O editor Droplet atualmente suporta JavaScript, HTML e CoffeeScript, com uma implementação em Python em desenvolvimento (Blanchard, 2017). Outras implementações dessa abordagem incluem o Blockly, uma ferramenta baseada em Python com foco em contextos de ciência de dados (Bart, Tibau, Kafura, Shaffer e Tilevich, 2017), o Tiled Grace, uma camada sobre a linguagem de programação Grace que suporta interações baseadas em blocos (Homer e Noble, 2017), o GP, uma reimagem do ambiente Scratch projetada para ser mais versátil (Mönig, Ohshima e Maloney, 2015), e o trabalho de Matsuzawa, Ohata, Sugiura e Sakai (2015), que criaram um ambiente Java bidirecional, um dos primeiros a estudar empiricamente essa abordagem e demonstrar seu potencial. Pesquisas estão revelando algumas das maneiras pelas quais os alunos aproveitam a bidirecionalidade desses ambientes, como a transição de texto para blocos a fim de introduzir novos comandos em um programa (Weintrop e Holbert, 2017).

Ao contrário dos ambientes bidirecionais, a abordagem de dupla modalidade combina características da programação em blocos e da programação baseada em texto em um único editor. A ideia por trás dessa estratégia é tentar aproveitar os pontos fortes de ambas as modalidades simultaneamente. Isso pode ser observado no editor baseado em frames para a linguagem Stride de Greenfoot, projetado para ser uma abordagem de programação em blocos orientada por teclado (Kölling, Brown & Altadmri, 2017). Pesquisas sobre essa abordagem mostraram que os alunos obtêm ganhos de aprendizado semelhantes aos de ambientes tradicionais, mas progredem mais rapidamente e apresentam menos problemas relacionados à sintaxe da linguagem (Price, Brown, Lipovac, Barnes & Kölling, 2016). Outras pesquisas com iniciantes trabalhando em ambientes de dupla modalidade revelaram que os aprendizes aproveitam aspectos de ambas as modalidades durante o trabalho, o que torna essa experiência distinta daquela obtida em um ambiente totalmente baseado em blocos ou totalmente baseado em texto (Weintrop & Wilensky, 2018). O desenvolvimento ativo desses ambientes demonstra a relevância atual do estudo proposto, que busca compreender como os ambientes baseados em blocos apoiam, ou não, os aprendizes em sua transição para línguas baseadas em texto.

2.3. Da programação baseada em blocos à programação baseada em texto

Com o aumento da popularidade dos ambientes de programação baseados em blocos em contextos educacionais formais, uma questão de crescente importância é saber se essa abordagem é eficaz na preparação dos alunos para futuras linguagens de programação baseadas em texto. Os primeiros trabalhos sobre programação gráfica (mas não baseada em blocos) encontraram poucas evidências de transferência bem-sucedida em iniciantes ao migrarem para linguagens de programação baseadas em texto, o que significa que os iniciantes ainda enfrentavam dificuldades com tarefas básicas de programação na nova linguagem, apesar do sucesso prévio com ferramentas gráficas (Scholtz & Wiedenbeck, 1990; Wiedenbeck, 1993). Resultados semelhantes foram documentados com o uso de ambientes contemporâneos baseados em blocos, com inúmeros estudos de caso documentando os desafios associados a...

transição (por exemplo, [Cliburn, 2008](#); [Garlick & Cankaya, 2010](#); [Powers et al., 2007](#)). [Kölling, Brown e Altadmri \(2015\)](#) documentam 13 questões distintas relacionadas a essa transição, que variam desde a necessidade de memorizar a sintaxe até a mudança para digitar comandos caractere por caractere em vez de blocos que tratam os comandos como objetos atômicos.

Ao mesmo tempo, existem estudos que mostram que os ganhos obtidos com ferramentas baseadas em blocos preparam os alunos para a programação textual posterior. [Armoni et al. \(2015\)](#) conduziram um estudo longitudinal para investigar esse tópico. Sua abordagem consistiu em analisar o desempenho de alunos em uma turma de programação textual do ensino médio, composta por alguns alunos que haviam cursado Scratch no ensino fundamental e outros sem experiência prévia em programação. Os pesquisadores então utilizaram o desempenho dos alunos no curso de programação do ensino médio para avaliar se a experiência com Scratch no ensino fundamental os ajudou nesse curso posterior. A análise resultante relatou pouca diferença quantitativa em seus desempenhos nas avaliações, mas identificou algumas áreas de conteúdo específicas em que os alunos com experiência prévia em Scratch superaram seus colegas (por exemplo, iteração de estruturas de programação). Os autores também descobriram que os alunos com experiência prévia em Scratch relataram níveis mais altos de autoeficácia e motivação para aprender a programar. [Grover, Pea e Cooper \(2015\)](#) utilizaram uma perspectiva de preparação para a aprendizagem futura para demonstrar que um currículo baseado em blocos pode preparar os alunos para a programação textual, incluindo resultados que mostram um bom desempenho dos alunos em questões conceituais de múltipla escolha em linguagens textuais após a conclusão de um currículo baseado em Scratch. Utilizando uma versão do Alice projetada para apoiar a transição e uma estratégia pedagógica que enfatizava essa transição, [Dann et al. \(2012\)](#) constataram que os alunos apresentaram melhor desempenho em cursos futuros de Java após a conclusão do currículo focado na transferência. Estudos subsequentes sobre a transição revelam o potencial dessa abordagem para auxiliar os alunos nessa transição (por exemplo, [Saito, Washizaki e Fukazawa, 2016](#); [Tabet, Gedawy, Alshikhabobak e Razak, 2016](#)). Esses estudos sugerem que ferramentas baseadas em blocos podem auxiliar os alunos em futuros contextos textuais, especialmente no que se refere ao conhecimento conceitual e à capacidade de criar programas bem-sucedidos. Contudo, a ausência de uma comparação controlada ou de um delineamento quase-experimental nesse trabalho anterior impede que os autores façam afirmações categóricas sobre o papel da modalidade no apoio a essa transição, independentemente do currículo ou da pedagogia. É essa questão, e as lacunas metodológicas associadas, que o presente trabalho busca abordar.

3. Materiais e métodos

3.1. Desenho do estudo e estratégia de coleta de dados

Este artigo busca responder à seguinte questão de pesquisa: Em aulas introdutórias de ciência da computação no ensino médio, como a modalidade utilizada para o ensino de programação introdutória (baseada em blocos versus baseada em texto) impacta os alunos na transição para uma linguagem de programação profissional baseada em texto? Para responder a essa questão, conduzimos um estudo quase-experimental durante as primeiras 15 semanas do ano letivo em duas turmas introdutórias de programação. As duas turmas seguiram o mesmo currículo, mas utilizaram versões diferentes dos mesmos ambientes de programação durante a primeira parte do curso. Nas primeiras cinco semanas do ano letivo, uma turma utilizou uma versão baseada em blocos do ambiente e a outra turma utilizou uma versão baseada em texto do mesmo ambiente. Mais detalhes sobre

Os ambientes são apresentados na próxima seção. Após a introdução de cinco semanas, ambas as turmas migraram para o ambiente de programação Java. Uma vez em Java, ambas as turmas utilizaram o mesmo ambiente de programação e seguiram o mesmo currículo. O delineamento do estudo previa que os alunos realizassem uma avaliação de conteúdo e uma pesquisa de atitudes no início, após cinco semanas, quando ocorreu a transição para Java, e novamente ao final das 15 semanas. As avaliações de conteúdo e de atitudes foram aplicadas em dias consecutivos durante o horário de aula e levaram aproximadamente 20 minutos para serem concluídas. Uma análise detalhada dos resultados comparativos das primeiras cinco semanas do estudo pode ser encontrada em [\(Weintrop & Wilensky, 2017a\)](#) e as versões completas de ambos os instrumentos utilizados neste estudo podem ser encontradas nos apêndices de [\(Weintrop, 2016\)](#).

Para a avaliação de conteúdo, os alunos foram solicitados a completar uma versão personalizada da Avaliação Computativa, desenvolvida especificamente para o estudo [\(Weintrop & Wilensky, 2015a\)](#). A avaliação consiste em 30 questões de múltipla escolha sobre os conceitos de programação abordados na parte introdutória do curso (incluindo lógica condicional e iterativa, variáveis e funções). A avaliação também incluiu questões relacionadas à compreensão geral do programa e questões não relacionadas à programação sobre algoritmos. A principal característica da Avaliação Computativa é que cada um dos pequenos programas pode ser apresentado em Snap!

Blocos, blocos de código de lápis ou texto de código de lápis ([Fig. 1a, b e c](#), respectivamente). As questões de compreensão pedem aos alunos que selecionem a descrição correta do programa apresentado; portanto, em vez de apenas descobrir a saída do programa, os alunos devem deduzir o propósito do conjunto de instruções. Observe que reconhecemos as limitações das questões de múltipla escolha para questões de compreensão de programas [\(Simon e Snowden, 2014\)](#), mas as incluímos na análise, pois representavam uma pequena parte da avaliação geral e foram mantidas constantes entre os grupos, tornando-as ainda úteis para nossas questões comparativas. Cada versão da avaliação solicitava

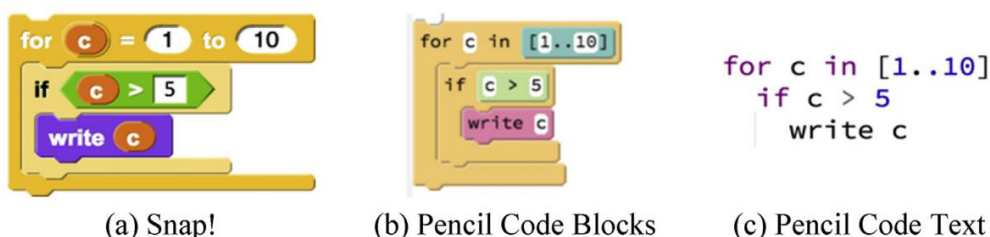


Figura 1. As três formas que os programas podem assumir na Avaliação Computativa: (a) Snap!, (b) Blocos de código a lápis, (c) Texto em código a lápis.

Os alunos deveriam responder a perguntas em todas as três modalidades de programação. A avaliação foi elaborada de forma que, em cada aplicação, os alunos respondessem a perguntas sobre todos os tópicos em todas as três modalidades. Além disso, três versões da avaliação foram construídas com diferentes padrões de pares pergunta-modalidade e aplicadas em cada um dos três intervalos de tempo. Isso resultou em um delineamento contrabalançado, no qual cada aluno respondeu a todas as perguntas em cada uma das três modalidades ao longo das 15 semanas, garantindo uma distribuição uniforme de conceito, condição e modalidade.

Juntamente com a avaliação de conteúdo, os alunos responderam a um questionário de atitudes no início, no meio e no final do estudo. As três versões do questionário aplicadas durante o estudo foram praticamente as mesmas, exceto por mudanças no tempo verbal (passado/futuro) e pela inclusão de perguntas adicionais nos questionários do meio e do final, solicitando aos alunos que refletissem sobre suas experiências na disciplina. O questionário foi baseado em itens do projeto Georgia Computes (Bruckman et al., 2009, pp. 86–90) e no Computing Attitudes Survey, que foi validado e é amplamente utilizado em pesquisas sobre educação em computação (Dorn & Elliott Tew, 2015; Tew, Dorn & Schneider, 2012). O questionário era composto por perguntas com escala Likert de 10 pontos, além de perguntas de resposta curta. Na análise apresentada neste artigo, focamos nas perguntas relacionadas à confiança e ao prazer, e em uma pergunta específica sobre o interesse em futuros cursos de ciência da computação.

Finalmente, como parte deste estudo, registramos os programas que os alunos escreveram e as mensagens de erro que receberam ao compilá-los. Ao programar em Java, antes de um programa ser executado, ele deve ser compilado usando o comando 'javac'. Se houver um erro no programa, uma chamada a 'javac' retornará um erro, impedindo a execução do programa. Para este estudo, registramos cada chamada a 'javac' feita pelos participantes, capturando o conteúdo do programa e a saída da chamada a 'javac' (incluindo quaisquer erros, se presentes). Para isso, adicionamos lógica ao console que os alunos usavam para executar seus programas. Para registrar esses eventos, desenvolvemos uma ferramenta que encapsulou o comando de compilação dos alunos com um script que executa a compilação, registrando também o conteúdo do programa do aluno e a saída do compilador, enviando ambos para um servidor remoto. Todo esse processo é invisível para o aluno. Essa abordagem para registrar os programas dos alunos se baseia em trabalhos anteriores que analisaram eventos de compilação como um meio de obter insights sobre as práticas de programação emergentes dos aprendizes (Jadud & Henriksen, 2009). A implementação específica utilizada neste trabalho foi baseada no projeto Git Data Collection (Danielak, 2014).

3.2. Pencil.cc e o currículo introdutório

Este estudo de pesquisa se baseia na aprendizagem de programação por alunos no mesmo ambiente de programação, porém utilizando diferentes modalidades – seja baseada em blocos ou baseada em texto. Para realizar este estudo, foi criada uma versão modificada do Pencil Code, chamada Pencil.cc. O Pencil Code é um ambiente de programação online que permite aos usuários alternar livremente entre versões baseadas em texto (Fig. 2a) e em blocos (Fig. 2b) de seus programas (Bau, Bau, Dawson e Pickens, 2015). Para suportar essa bidirecionalidade, as duas modalidades de programação são isomórficas, ou seja, tudo o que pode ser feito em uma interface também pode ser feito na outra. Diferentemente do Pencil Code, o Pencil.cc impede que os alunos alternem entre as duas modalidades. Em vez disso, os alunos usam apenas a interface baseada em blocos ou apenas a interface baseada em texto. Assim, durante as cinco semanas do estudo, os alunos foram apresentados à programação usando uma versão do Pencil.cc baseada em blocos ou uma versão baseada em texto. Isso significa que os alunos de uma turma programaram usando o mecanismo de arrastar e soltar da interface baseada em blocos, enquanto os alunos da outra turma criaram programas digitando comandos caractere por caractere. Além da modalidade de programação, tudo o mais nas duas versões do ambiente de programação é idêntico, incluindo a linguagem de programação (incluindo palavras-chave e sintaxe), o ambiente de execução visual e os recursos de programação e outros recursos de suporte. Para ambas as versões do Pencil.cc, a linguagem de programação subjacente utilizada foi o CoffeeScript. O CoffeeScript foi escolhido por ser sintaticamente simples, ter uma base ativa de usuários profissionais e ser suficientemente diferente do Java para manter a transição entre os ambientes significativa.

A interface baseada em blocos do Pencil.cc inclui muitas das características da programação em blocos que pesquisas identificaram como sendo percebidas pelos alunos como produtivas para a programação (Weintrop & Wilensky, 2015b). Isso inclui uma paleta de blocos organizada conceitualmente e navegável (a porção mais à esquerda da Fig. 2b), dicas visuais sobre como e onde os blocos podem ser usados e o mecanismo de composição por arrastar e soltar. Os alunos na condição baseada em texto do estudo usaram um editor de texto que inclui diversos recursos de apoio.

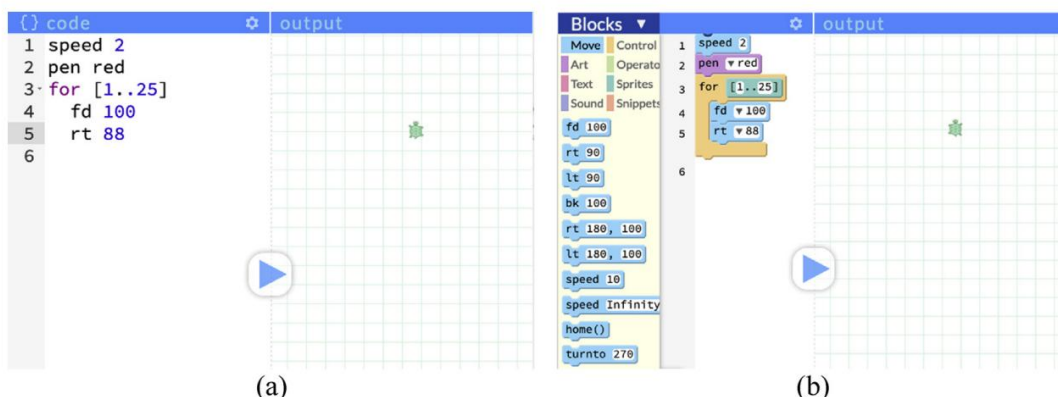


Figura 2. As versões do Pencil.cc baseadas em texto (a) e em blocos (b) utilizadas neste estudo.

O Pencil.cc é um programa padrão para ferramentas de autoria de programação, incluindo realce de sintaxe e verificação básica de erros em tempo de compilação. O objetivo na criação do Pencil.cc foi isolar a modalidade de programação para entender se e como a forma como os alunos são inicialmente apresentados à programação (ou seja, baseada em blocos versus baseada em texto) afeta sua capacidade de transição para linguagens textuais convencionais.

O currículo introdutório seguido pelos alunos durante as primeiras cinco semanas foi uma versão modificada do livro "Beauty and Joy of Computing" (Garcia et al., 2015). Atividades adicionais foram criadas para aproveitar os recursos do Pencil.cc, que se baseiam na tradição da programação construcionista (Harvey, 1997; Papert, 1980). Na criação do currículo, buscou-se garantir que os alunos tivessem liberdade criativa em cada tarefa. Assim, as soluções dos alunos variaram bastante dentro das tarefas. O currículo de cinco semanas abordou os seguintes tópicos: variáveis, lógica condicional, lógica iterativa e funções. Cada área de conteúdo incluiu tarefas visuais que solicitavam aos alunos que controlassem uma tartaruga na tela (no espírito das atividades do Logo e do Scratch), bem como atividades de processamento de texto sem componente gráfico. Os tipos de tarefas no currículo foram equilibrados para não privilegiar uma condição em detrimento da outra, caso uma modalidade fosse mais adequada a um tipo específico de atividade. O currículo utilizado no estudo está disponível no Apêndice A de (Weintrop, 2016).

3.2.1. Materiais introdutórios de Java

Após as cinco semanas introdutórias, os alunos de ambos os grupos passaram a estudar Java. Todos os alunos utilizaram o mesmo editor de texto básico para as tarefas de programação em Java, que não incluía recursos comuns de editores de programação, como realce de sintaxe ou autocompletar. A professora considerou pedagogicamente valioso que os alunos trabalhassem em um editor de texto básico, que ela já havia utilizado com sucesso neste curso em anos anteriores. O curso seguiu o livro-texto "Java Concepts: Early Objects" (Horstmann, 2012). Durante as dez semanas dedicadas à linguagem Java, os alunos tiveram contato com entrada/saída básica, variáveis, tipos de dados, objetos e funções. Embora não haja sobreposição completa de conteúdo entre o currículo introdutório e as dez primeiras semanas de Java, existem conceitos comuns a ambos, principalmente variáveis e funções.

3.2.2. Contexto e participantes

Este estudo foi realizado em uma escola pública urbana no Meio-Oeste americano, que atende a quase 4.000 alunos. A escola é racial e socioeconomicamente diversa (44% hispânicos, 33% brancos, 10% asiáticos, 9% negros e 4% multirraciais/outras; 58,6% dos alunos são de famílias economicamente desfavorecidas). A escola é uma instituição de admissão seletiva, o que significa que é destinada a alunos com bom desempenho acadêmico que precisam obter um bom resultado em uma prova escrita para serem admitidos. O distrito escolar implementou medidas para garantir que a composição das escolas de admissão seletiva seja representativa da população urbana da qual provém, de modo que os alunos sejam admitidos com base em seu desempenho na prova em relação a outros alunos de sua escola atual, e não em relação a outros alunos da cidade. O resultado dessa decisão é um corpo discente mais diverso (tanto socioeconomicamente quanto geograficamente) do que o obtido com abordagens alternativas para a seleção de alunos. Trabalhar em uma escola de admissão seletiva não era o ideal para este trabalho e apresenta algumas limitações, mas era necessário, pois poucas escolas públicas têm várias turmas do mesmo curso de ciência da computação ou um professor experiente o suficiente para lecionar para diferentes turmas usando variações da mesma ferramenta de programação. A população estudantil da qual os participantes foram selecionados representa um conjunto representativo de alunos de alto desempenho.

O delineamento quase-experimental foi realizado em duas turmas de um curso introdutório de programação já existente. Em anos anteriores, o curso começava a ensinar Java no primeiro dia de aula. Para este estudo, a introdução ao Java foi adiada para a sexta semana de aula, com as cinco primeiras semanas dedicadas ao uso do Pencil.cc e do currículo personalizado descrito anteriormente. Cada uma das duas turmas tinha 30 alunos, cada um dos quais recebeu um laptop para usar durante o curso. As duas turmas foram ministradas pela mesma professora, na mesma sala de aula, em períodos consecutivos, o que nos permitiu controlar o efeito da professora e do ambiente. A professora era graduada em educação técnica e treinamento corporativo e estava em seu oitavo ano de docência (terceiro naquela instituição).

A disciplina de Introdução à Programação é oferecida como eletiva, ou seja, os alunos optam por se matricular nela. Historicamente, a disciplina atrai alunos de diversas origens raciais e geralmente tem mais alunos do sexo masculino do que do sexo feminino. Este estudo incluiu um total de 60 alunos, com todos os alunos das duas turmas incluídos no conjunto de participantes potenciais. A escolha de duas turmas de 30 alunos cada se deu porque 30 é frequentemente considerado o número mínimo de participantes para análises estatísticas, o que nos permite analisar as condições de forma independente (Cohen, Manion & Morrison, 2007). Além disso, a escola oferecia apenas 3 turmas da disciplina, todas envolvidas em estudos de pesquisa, e incluir uma segunda turma teria introduzido novos fatores de confusão no desenho e na análise do estudo. A distribuição racial autodeclarada dos alunos da turma foi a seguinte: 41% brancos, 27% hispânicos, 11% asiáticos, 11% multirraciais e 10% negros. Entre os participantes, um idioma diferente do inglês é falado em 47% dos lares. As turmas eram compostas por 49 alunos do sexo masculino (25 na turma de Blocos e 24 na turma de Texto) e 11 alunas do sexo feminino (5 na turma de Blocos e 6 na turma de Texto). As duas turmas incluíam alunos de todos os quatro anos do ensino médio (9 do primeiro ano, 9 do segundo ano, 16 do terceiro ano e 26 do último ano). A disparidade de gênero nessas salas de aula é problemática, mas o recrutamento de alunos para as turmas utilizadas no estudo estava fora do controle dos pesquisadores.

4. Resultados

Este artigo busca compreender como a modalidade de programação introdutória (baseada em blocos versus baseada em texto) impacta os aprendizes quando estes progredem para o aprendizado de uma linguagem de programação profissional baseada em texto. Para este trabalho, operacionalizamos o conceito de "impacto" analisando três dimensões inter-relacionadas do aprendizado de programação: compreensão conceitual, atitudes em relação à programação e práticas de programação. Ao focar nesses três aspectos, buscamos compreender como o conhecimento, as disposições e as práticas desenvolvidas com o uso de ferramentas introdutórias são transferidas para o aprendizado de programação com uma linguagem profissional baseada em texto. Utilizamos esses três aspectos como meio de compreender diferentes dimensões do complexo ato de aprender a programar. Assim, esta seção apresenta três maneiras distintas de compreender como a modalidade introdutória impacta a experiência de aprendizado do iniciante à medida que ele transita do ensino introdutório para o ensino profissional.

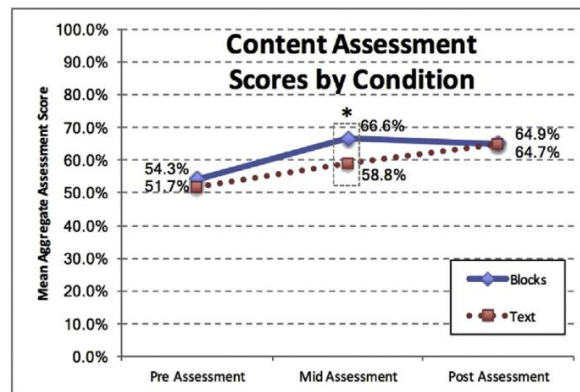


Figura 3. As pontuações médias dos alunos nas duas condições nas três aplicações (Pré, Intermediária e Pós) da Avaliação Computativa.

Ferramentas profissionais.

Esta seção começa com uma apresentação dos resultados da compreensão conceitual do conteúdo de programação pelos alunos, por meio da análise do desempenho deles na Avaliação Computativa. Em seguida, apresentamos uma análise sistemática das respostas aos questionários de opinião aplicados. Finalmente, apresentamos uma análise das práticas de programação dos alunos com base na análise das chamadas ao javac. Ao longo desta seção, relatamos brevemente os resultados das cinco primeiras semanas do estudo, já publicados anteriormente, para contextualizar, mas nos concentramos especificamente nos resultados a partir da 5ª semana. Além disso, dada a natureza multidimensional da análise, apresentamos aqui breves discussões sobre os resultados específicos de cada análise, antes de fornecer uma análise abrangente das diferentes fontes de dados na seção de discussão subsequente.

4.1. Resultados de aprendizagem

Esta seção apresenta uma análise das pontuações das avaliações de conteúdo pré, intermediárias e pós-teste por condição. A Figura 3 mostra as pontuações dos alunos em cada uma das três administrações.

Começamos por apresentar resultados previamente publicados, referentes às pontuações na avaliação de conteúdo aplicada no início do ano letivo e no meio do estudo (Weintrop & Wilensky, 2017a). Por condição, as pontuações médias foram de 51,7% (DP 14,5%) para a condição Texto e 54,3% (DP 12,2%) para a condição Blocos, uma diferença que não se mostrou estatisticamente significativa: $t(57) = 0,78$, $p = 0,44$, $d = 0,20$ (nota: os cálculos que mostram um número de participantes inferior a 60 devem-se às ausências dos alunos, para as quais não foi possível a realização de provas de reposição). Esta análise demonstra que não houve diferença entre as duas turmas em relação ao conhecimento prévio de programação no início do estudo. Após cinco semanas de trabalho no ambiente introdutório, surgiu uma diferença entre as duas condições. Analisando a avaliação intermediária, os alunos na condição Texto obtiveram uma pontuação média de 58,8% (DP 14,6%) e os alunos na condição Blocos obtiveram uma média de 66,6% (DP 13,4%). Um teste t mostrou que essa diferença é estatisticamente significativa ($t(53) = 2,03$, $p = 0,04$, $d = 0,58$). Isso significa que, após 5 semanas, os alunos que aprenderam a programar em um ambiente baseado em blocos tiveram um desempenho significativamente melhor em uma avaliação de conteúdo do que os colegas que usaram um ambiente isomórfico baseado em texto.

Após a avaliação intermediária, os alunos passaram imediatamente para Java e dedicaram as 10 semanas seguintes à programação em Java antes de realizarem a Avaliação Computativa pela terceira e última vez. Na avaliação de conteúdo posterior, não foi encontrada diferença estatística entre as duas condições ($t(57) = 0,014$, $p = 0,99$), com os alunos do grupo Blocos obtendo uma média de 64,9% (DP 13,5%) e os alunos do grupo Texto alcançando 64,7% (DP 14,0%). Ao analisarmos a evolução do desempenho dos alunos entre as avaliações intermediária e posterior e compararmos os resultados entre as duas condições, constatamos que a diferença nos ganhos obtidos pelos alunos entre as duas condições foi estatisticamente significativa ($t(52) = 2,58$, $p = 0,01$, $d = 0,70$).

Esta análise revela que os alunos do grupo de programação em blocos obtiveram ganhos maiores durante o período introdutório de cinco semanas em relação ao desempenho na Avaliação Computativa. Após a transição para Java, os alunos do grupo de blocos não apresentaram melhorias, enquanto o grupo de texto observou uma melhora incremental. Isso significa que, ao longo das 15 semanas do estudo, todos os alunos começaram e terminaram no mesmo nível na avaliação de conteúdo; no entanto, os dois grupos seguiram caminhos diferentes durante esse período.

Uma possível explicação é que houve pouca transferência entre os ambientes para os alunos provenientes da condição baseada em blocos, de modo que seu desempenho se estabilizou, enquanto os alunos que passaram as cinco semanas anteriores trabalhando no ambiente introdutório baseado em texto experimentaram alguma transferência que os ajudou a continuar desenvolvendo sua compreensão conceitual dos conceitos de programação à medida que avançavam para Java. Uma segunda possível explicação para essa descoberta é que existe um efeito teto para os aprendizes e que a condição de Blocos atingiu esse teto mais rapidamente do que a condição de Texto. Em outras palavras, os alunos no ambiente que permitia a composição por arrastar e soltar conseguiram compreender os conceitos em questão mais rapidamente, enquanto a condição de Texto levou mais tempo para compreender a atividade de programação antes de atingir o teto associado ao currículo trabalhado pelos alunos. A descoberta de que ambientes de aprendizagem baseados em blocos permitem que os alunos aprendam mais rapidamente foi demonstrada em alguns pequenos estudos em ambientes informais (Price & Barnes, 2015, pp. 91–99), o que sugere que esse pode ser um fenômeno maior e mais robusto.

4.2. Resultados atitudinais

Parte da compreensão e avaliação do impacto do trabalho com diferentes modalidades durante a parte introdutória do curso envolve investigar como as atitudes e percepções de programação formadas durante o uso dessas modalidades persistiram ou mudaram à medida que os alunos passaram a estudar Java. Como mencionado anteriormente, nesta seção, apresentamos os resultados de nossa análise de duas medidas atitudinais compostas (confiança e prazer) e uma questão independente relacionada ao interesse em futuros cursos de ciência da computação. A decisão de focar nessas dimensões específicas de atitude se deve à sua associação com a probabilidade de um aluno buscar futuras oportunidades de aprendizado em ciência da computação, especialmente no que se refere a alunos de populações historicamente sub-representadas na área da computação (American Association of University Women, 1994; Margolis & Fisher, 2003; Tavani & Losh, 2003). Além disso, confiança e prazer são frequentemente utilizados em estudos que investigam o design de ambientes de programação e aprendizagem (por exemplo, Bishop-Clark, Courte e Howard, 2006; McDowell, Werner, Bullock e Fernald, 2006; Rodriguez Corral et al., 2019). Ao analisar mudanças nas pontuações ao longo do tempo no mesmo conjunto de alunos (por exemplo, entre as pesquisas intermediária e final para a condição de Blocos), realiza-se um teste de Wilcoxon para postos sinalizados (apresentado como estatística Z). A natureza ordinal das respostas Likert e a natureza não paramétrica do teste tornam este um teste apropriado para esses cálculos de amostras pareadas. Nos casos em que a análise compara as duas condições entre si, realiza-se um teste de Wilcoxon para soma de postos (apresentado como estatística U). Este teste é utilizado porque as duas amostras são independentes e os dados subjacentes são ordinais e não paramétricos (Fay e Proschan, 2010). Observe que os valores nesta seção são apresentados na mesma escala do eixo y, mas não abrangem a mesma faixa, o que significa que podem ser comparados entre si, embora o significado das posições nos gráficos seja diferente. É importante notar também que os eixos y dos valores apresentados nesta seção não começam em zero; isso foi feito para tornar as diferenças quantitativas mais legíveis.

4.2.1. Confiança na capacidade de programação

A primeira dimensão atitudinal discutida é a confiança percebida pelos alunos em sua capacidade de programação. A pontuação composta de confiança é a combinação das duas afirmações da escala Likert: "Serei bom em programação" e "Me sairei bem neste curso" (nota: o tempo verbal das afirmações mudou entre as aplicações). Essas questões apresentaram valores de alfa de Cronbach de 0,82 na pré-avaliação, 0,80 na avaliação intermediária e 0,88 na pós-avaliação, que atendem ao limite de 0,8 frequentemente citado como o nível mínimo de aceitabilidade para fins de pesquisa (Streiner, 2003). A Figura 4a mostra a medida de confiança agregada nos momentos de pré-avaliação, avaliação intermediária e pós-avaliação.

Ao analisar os três momentos distintos em que o questionário foi aplicado, não encontramos diferença significativa na confiança entre as condições (Pré: $U = 353,5$, $p = 0,30$; Intermediário: $U = 395$, $p = 0,78$; Pós: $U = 307,5$, $p = 0,08$). Ao realizar o teste de Wilcoxon para amostras pareadas, a fim de verificar mudanças dentro dos grupos entre os momentos de aplicação, encontramos uma diferença significativa apenas na condição de Blocos entre os questionários Intermediário e Pós. Essa diferença demonstra que os alunos que trabalharam por 5 semanas na condição baseada em blocos apresentaram uma confiança significativamente menor após 10 semanas de programação em Java ($Z = 46$, $p = 0,05$). Observe que, apesar de haver uma diferença numérica entre as condições no início do estudo, essa diferença não foi estatisticamente significativa, assim como a mudança na confiança entre as aplicações Pré e Intermediária do questionário. Em conjunto, os dados mostram que, comparativamente, a modalidade não teve um impacto significativo na confiança dos alunos. Ao relacionarmos esses resultados com as descobertas da seção anterior sobre desempenho de conteúdo, observamos que os alunos do grupo Texto apresentam um leve (embora não significativo) aumento na confiança, ao mesmo tempo em que suas notas nos testes melhoram, enquanto os alunos do grupo Blocos apresentam uma diminuição na confiança juntamente com uma queda nas notas da avaliação de conteúdo. A conclusão que podemos tirar desta análise é que a modalidade, por si só, não parece afetar a confiança dos alunos do ensino médio em sua capacidade de programação.

Analisando apenas a trajetória para as condições de Blocos, o aumento na confiança dos alunos poderia explicar outras descobertas que mostram uma maior retenção para alunos que usam esses tipos de ferramentas gráficas em seu primeiro curso de ciência da computação (Cliburn, 2008; Johnsgard & McDonald, 2008), mas potencialmente coloca em questão a eficácia de tal abordagem para preparar os alunos para o aprendizado futuro de ciência da computação, já que os ganhos em relação à confiança não persistem.

4.2.2. Prazer em programar

A segunda dimensão atitudinal analisada na pesquisa busca compreender se os alunos gostavam de programação e, em caso afirmativo, como isso acontecia.

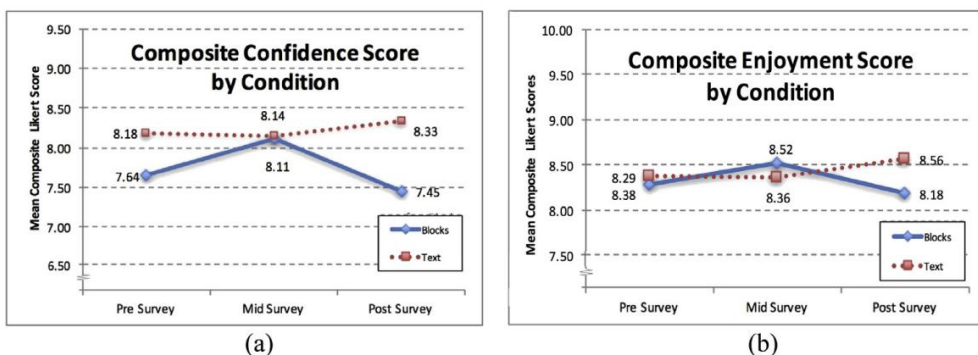


Figura 4. Pontuações compostas de confiança (a) e prazer (b) dos alunos em programação em três momentos do estudo.

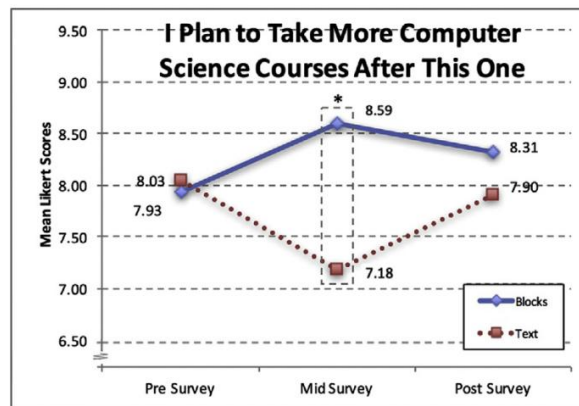


Figura 5. As pontuações médias das respostas, agrupadas por condição, para a afirmação: Pretendo fazer mais cursos de ciência da computação depois deste.

As condições variaram tanto durante o uso das ferramentas introdutórias quanto durante o tempo dedicado ao Java. A pontuação geral de satisfação é uma composição das respostas às três perguntas a seguir: Programar é divertido, eu gosto de programar e estou animado com este curso.

Foi realizado um teste Alfa de Cronbach nessas três questões e constatou-se um nível suficiente de correlação (Pré: $\bar{y} = 0,79$, Meio: $\bar{y} = 0,84$, Pós: $\bar{y} = 0,89$).

Ao analisar as mudanças do período Pré para o Meio e do Meio para o Pós, mostradas na Figura 4b, observa-se pouca alteração quantitativa, sem diferenças estatisticamente significativas entre os períodos de tempo dentro dos grupos ou no mesmo período de tempo entre os grupos.

Essa ausência de diferença leva à conclusão de que a modalidade não afeta a percepção de prazer em alunos do ensino médio. É importante lembrar que os alunos trabalharam com o mesmo currículo, independentemente da modalidade. Em outras palavras, os alunos que trabalharam na interface baseada em blocos receberam as mesmas tarefas que aqueles que trabalharam com texto. Essa ausência de diferença significativa sugere que o maior prazer em programar encontrado em outros estudos que utilizaram ferramentas baseadas em blocos (por exemplo, Wilson & Moffat, 2010) pode estar mais relacionado ao currículo utilizado ou ao contexto em que os alunos programaram do que à própria modalidade.

4.2.3. Interesse em futuros estudos de Ciência da Computação

O resultado final da pesquisa de atitudes apresentado nesta análise questiona o interesse dos alunos em buscar futuras oportunidades de aprendizado em ciência da computação. A pesquisa solicitou que os alunos respondessem, em uma escala Likert de 10 pontos, à seguinte pergunta: "Pretendo fazer mais cursos de ciência da computação depois deste". As respostas dos alunos nos três momentos da pesquisa, agrupadas por condição, são mostradas na Figura 5.

Assim como observado nas pontuações de conteúdo apresentadas na Figura 3, ao analisarmos o interesse dos alunos em futuros cursos de ciência da computação, notamos que os alunos em ambas as condições iniciam e terminam o estudo no mesmo ponto, com uma diferença significativa surgindo no ponto médio do estudo. Nas primeiras cinco semanas do estudo, os participantes da condição Blocos demonstraram maior interesse em cursos de ciência da computação, enquanto os alunos da condição Texto demonstraram menor interesse. No ponto médio do estudo, as duas condições apresentaram níveis de interesse significativamente diferentes em futuros cursos de ciência da computação ($U = 264,5$, $p < 0,05$). Após 10 semanas de trabalho com Java, a tendência se inverte, com os alunos provenientes do ambiente Blocos demonstrando menor interesse, enquanto os alunos provenientes da condição Texto demonstraram maior interesse.

Os resultados desta análise são particularmente interessantes, pois têm implicações potenciais para o momento estratégico da transição de blocos para texto em contextos formais. Se os criadores de uma sequência de cursos introdutórios de ciência da computação planejam que os alunos usem inicialmente um ambiente baseado em blocos com o objetivo de, em algum momento, fazer a transição para a programação baseada em texto, os dados apresentados na Figura 5 sugerem que existem momentos melhores e piores para agendar essa transição. Por exemplo, se a sequência de cursos for definida de forma que o primeiro curso seja totalmente em blocos e o segundo utilize apenas linguagens baseadas em texto (uma abordagem comum), esses dados sugerem que haverá uma alta taxa de matrícula no segundo curso, mas que o interesse dos alunos diminuirá rapidamente. Esse padrão coincide com relatos de educadores que implementaram essa abordagem (Cliburn, 2008). A possível desvantagem dessa abordagem é que, se a introdução à programação baseada em texto ocorrer no início do curso, é mais provável que os alunos se transfiram para outra disciplina ou abandonem o curso. Alternativas a essa abordagem seriam realizar a transição dentro do próprio curso, de modo que o professor tenha estabelecido uma relação de confiança com os alunos e empregando técnicas explícitas de ponteamento (Dann et al., 2012), ou intercalar o ensino baseado em blocos e o ensino baseado em texto ao longo do ano (Powers et al., 2007). É importante notar também que existem outros motivos para usar uma sequência que priorize os blocos, apesar dessa constatação, como os maiores ganhos de aprendizagem (conforme discutido na seção 4.1) ou a priorização da exposição a conceitos fundamentais e da introdução facilitada em detrimento da preparação para o aprendizado futuro e a transição para a programação baseada em texto, que é enfatizada neste estudo.

4.3. Resultados das práticas de programação

A dimensão final da nossa operacionalização do impacto investiga as diferenças nas práticas de programação entre alunos que utilizam ferramentas baseadas em blocos e aqueles que tiveram uma introdução à programação baseada em texto. Ao contrário das duas seções anteriores de Resultados, que utilizaram questionários e avaliações escritas, esta seção analisa os programas que os alunos escreveram e como os escreveram para identificar melhor os potenciais impactos da modalidade introdutória. Esta seção examina algumas dimensões diferentes da prática de

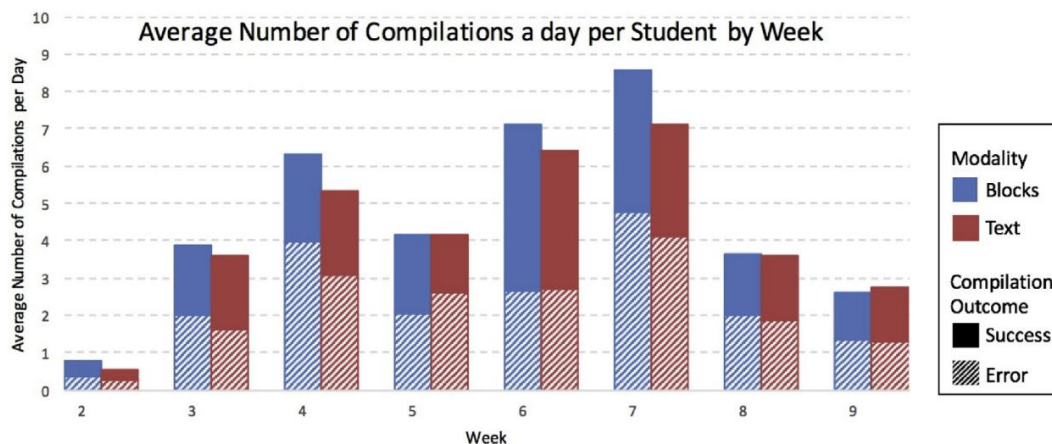


Figura 6. Número médio de chamadas javac por aluno por dia, agrupadas por semana. A parte sólida de cada barra representa chamadas bem-sucedidas; a parte listrada, chamadas bem-sucedidas. Algumas partes representam chamadas errôneas.

programação que foi investigada em outros trabalhos na literatura sobre educação em ciência da computação para compreender o processo de aprendizagem para programar: comportamentos de compilação e execução (Jadud, 2005; Jadud & Henriksen, 2009; Vihavainen, Luukkainen, & Ihtantola, 2014, pp. 21–26), tipos e frequência de erros (Altadmri & Brown, 2015; Jackson, Cobb, & Carver, 2005; Jadud, 2005) e alterações realizadas entre execuções sequenciais (Berland, Martin, Benton, Petrick Smith, & Davis, 2013; Blikstein et al., 2014; Piech, Sahami, Koller, Cooper, 2014) & Blikstein, 2012). Nesta seção, utilizamos os dados gerados quando os alunos tentam compilar o programa Java usando o javac. comando como meio de compreender as práticas de programação emergentes.

4.3.1. Diferenças na frequência com que os alunos tentaram executar seus programas

Os alunos do grupo Blocks executaram o comando javac uma média de 142,3 vezes (DP 67,1) ao longo das dez semanas. Durante o mesmo período, os alunos do grupo Texto chamaram o comando javac 130,9 vezes (DP 61,1), uma diferença que não é estatisticamente significativa. significativo ($t(56) = 0,67$, $p = 0,50$). Em outras palavras, não houve diferença significativa no número de chamadas ao javac com base no modalidade introdutória utilizada pelos alunos. A Figura 6 mostra o número médio de compilações por aluno por dia nas três modalidades. Condições detalhadas por semana. Este gráfico inclui tanto compilações bem-sucedidas (partes sólidas de cada barra) quanto chamadas que resultou em um erro (a parte listrada de cada barra).

Embora o número de chamadas por semana varie ao longo das dez semanas devido às variações nas atividades em sala de aula, durante o período de dez semanas, a média de chamadas por semana se mantém constante. No estudo, observamos pouca diferença entre as duas condições. Em conjunto, isso demonstra que os alunos nas duas condições tiveram desempenhos semelhantes. Em seguida, analisamos as práticas de programação em termos da frequência com que tentavam executar seus programas e, para verificar se existem... diferenças na propensão a erros desses programas.

4.3.2. Diferenças nas taxas de sucesso das compilações

Se um aluno cometer um erro de sintaxe em seu programa, uma chamada à função javac retornará uma resposta de erro informando-o sobre o erro. No currículo de dez semanas, os alunos que utilizaram o ambiente introdutório baseado em texto apresentaram uma média de 70,26 erros (DP 38,2), enquanto Os alunos provenientes da condição introdutória baseada em blocos cometeram, em média, 75,7 erros no total (DP 34,3). Em termos percentuais, esse valor corresponde a... No grupo de estudo com o comando 'javac', os alunos da condição Texto apresentaram erros em 53,7% das chamadas, em comparação com 53,2% para os alunos da condição Blocos. A diferença nas taxas de erro não é estatisticamente significativa ($t(56) = 0,58$, $p = 0,57$). As porções listradas das colunas na Figura 6 mostram a número médio de chamadas javac malsucedidas por semana. Se analisarmos esses dados para observar a diferença por aluno, novamente vemos algo semelhante. padrões entre as duas condições. A Tabela 1 fornece uma visão geral da frequência de compilações reprovadas por aluno, bem como Informações sobre o número de erros por chamada javac com falha, discriminadas por condição.

Aqui, novamente, vemos pouca diferença entre as duas condições. Isso nos leva a concluir que as cinco semanas passadas no ambiente introdutório baseado em texto não prepararam melhor os alunos para a programação posterior em uma linguagem profissional baseada em texto. em relação à criação de programas menos propensos a erros. Ao mesmo tempo, isso não afetou negativamente a capacidade dos alunos de escrever programas corretos. Os programas, em comparação com aqueles que possuem experiência prévia em programação, estão situados em um ambiente baseado em blocos.

Tabela 1

Padrões descritivos de alto nível de falhas de compilação e erros ao longo das 10 semanas.

	Chamadas javac com falha por aluno	Erros de compilação por aluno	Erros de compilação por chamada javac com falha
Blocos	75,11	165,78	2,23
Texto	69,55	164,26	2,21

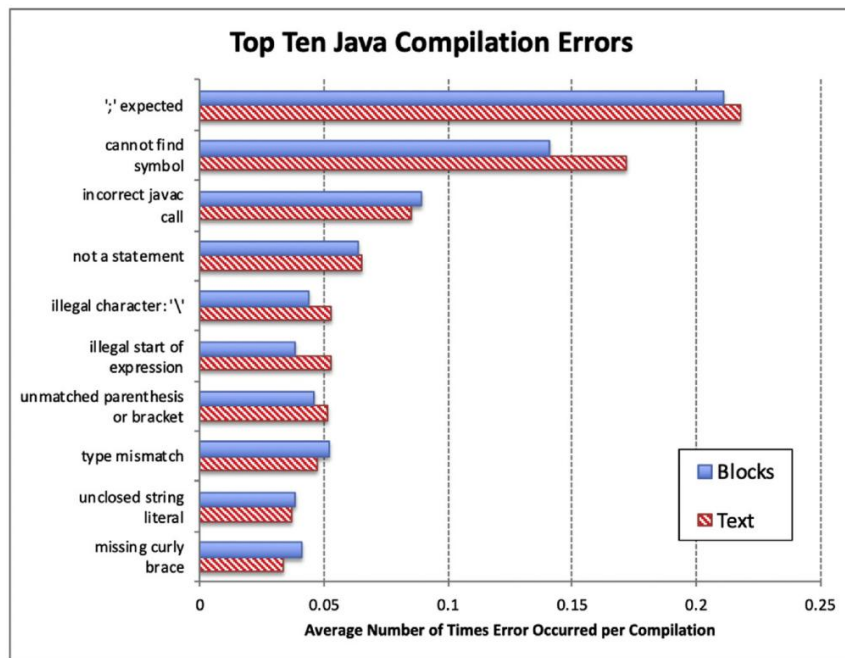


Figura 7. Os dez erros de Java mais frequentes, agrupados por condição.

4.3.3. Diferenças nos tipos de erros encontrados

Assim como podemos extrair informações de padrões encontrados em chamadas de compilação bem-sucedidas e errôneas, também podemos analisar os tipos dos erros de programação que os alunos encontraram como outra possível fonte de diferenças entre programação baseada em blocos e programação baseada em texto. introduções a serem emergentes. Esta seção relata os tipos de erros encontrados e sua frequência para verificar se existem padrões sistemáticos. diferenças baseadas na modalidade de introdução dos alunos. Infelizmente, o compilador Java muitas vezes não fornece (e às vezes não pode fornecer) mensagens de erro significativas para o programador (ou para pesquisadores que tentam entender o comportamento de programadores iniciantes). Por exemplo, um A ausência do caractere ";" pode resultar na mensagem de erro "esperava-se ";" na linha 11" ou na mensagem genérica "não é uma instrução". Para tornar a análise mais significativa, os erros foram agrupados em tipos de erro definidos de forma mais ampla, com uma descrição detalhada de como esses erros ocorreram. Os tipos de erros compilados podem ser encontrados em (Weintrop, 2016). A Figura 7 mostra os dez erros mais frequentes agrupados por condição. Os valores neste gráfico são relatados por compilação para controlar as diferenças na frequência com que os alunos compilaram seus programas.

O erro mais comum foi: "',' esperado", que ocorre quando os alunos se esquecem de terminar uma frase com um ponto e vírgula. Requisito sintático do Java. O segundo erro mais comum, "símbolo não encontrado", ocorre quando os alunos tentam usar uma variável antes de ter sido definido. Esse padrão coincide com pesquisas anteriores que analisaram padrões de erros de programação de iniciantes, como Jadud (2005) e Flowers et al. (2004) identificaram esses dois erros como os mais frequentemente encontrados por programadores iniciantes em aprendizado. Java. Combinados com a análise anterior, esses dados mostram que os alunos nas duas condições não apenas cometeram erros na mesma proporção. Mas quando eles tinham erros de sintaxe em seus programas, eram sempre do mesmo tipo. A conclusão desta análise é que A modalidade introdutória não influenciou o tipo de erros que os alunos cometeram após a transição para a linguagem de programação Java.

4.3.4. Diferenças na quantidade de código adicionado entre compilações bem-sucedidas

Além dos padrões de compilação, também estamos interessados em como se cria um programa. Por exemplo, o aprendiz... Escrever blocos maiores de código e depois ver o que acontece? Ou o aluno faz pequenas edições incrementais até chegar a um código funcional? programa? A ideia é investigar se a modalidade introdutória moldou a prática de composição em relação à prática de autoria. programas. Para isso, analisamos a magnitude das alterações feitas nos programas entre execuções consecutivas. Para medir o tamanho do Para detectar mudanças, utilizamos a distância de Levenshtein entre os textos dos dois programas (Levenshtein, 1966). A distância de Levenshtein captura

Tabela 2

A frequência de compilações bem-sucedidas com uma dada distância de Levenshtein em relação à última compilação bem-sucedida do mesmo programa.

	Distância de Levenshtein								
	012345–10			11–25			26–100		
Blocos	7,00	3,37	5,70	1,33	2,37	4,30	3,52	6,56	3,33
Texto	6,16	3,00	5,58	1,23	2,13	3,77	3,48	5,87	2,55

O número mínimo de edições de um único caractere (ou seja, inserções, exclusões ou substituições) necessárias para transformar uma string em outra. A Tabela 2 mostra os resultados desta análise. As colunas representam a distância de Levenshtein entre programas consecutivos compilados com sucesso, enquanto as células mostram o número médio de ocorrências dessa distância por aluno. Quanto menor o número, menos vezes um programa com essa distância da compilação bem-sucedida anterior foi executado por um aluno. Por exemplo, a coluna mais à esquerda, que contém números, mostra que, em média, os alunos na condição Blocos compilaram um programa idêntico ao último programa compilado 7,00 vezes ao longo das 10 semanas, enquanto na condição Texto isso ocorreu 6,16 vezes.

Os alunos do grupo Texto fizeram menos alterações significativas em seus programas e também executaram seus programas novamente sem fazer alterações com menos frequência do que os alunos do outro grupo. Esses números confirmam o que foi dito nas seções anteriores, ou seja, não há diferença sistemática na magnitude das alterações nos programas entre execuções consecutivas nos dois grupos após a transição para Java. Além disso, vale ressaltar que esses dados também nos permitem rejeitar a explicação de que o menor número total de complicações no grupo Texto foi resultado de os alunos terem feito conjuntos maiores de alterações entre as execuções.

5. Discussão

A principal contribuição deste trabalho é a descoberta de que, após observarem diferenças entre alunos que trabalham em ambientes introdutórios baseados em blocos e em texto, essas diferenças desaparecem depois que os alunos fazem a transição para um ambiente profissional baseado em texto. Essa ausência de diferença se verifica nas dimensões conceitual, atitudinal e de práticas de programação. Trata-se de uma descoberta significativa, visto que currículos e ambientes introdutórios de computação voltados para iniciantes estão se tornando cada vez mais comuns, tornando importante compreender os pontos fortes (como uma introdução intuitiva e acessível à área) e as limitações (como a falta de transferência imediata do conhecimento ou das práticas adquiridas) do ambiente de programação utilizado. Uma possível interpretação desses resultados é que, se o objetivo é preparar os alunos para programar em ambientes de programação convencionais baseados em texto, então as ferramentas baseadas em blocos podem não ser mais úteis do que as linguagens introdutórias baseadas em texto historicamente utilizadas. Contudo, essa interpretação ignora abordagens pedagógicas e de design que poderiam facilitar uma transição mais bem-sucedida para linguagens de programação profissionais. Utilizamos essas implicações para estruturar nossa seção de discussão antes de concluir com alguns pontos de discussão mais amplos.

5.1. Implicações para a pedagogia

Uma das decisões de projeto deste estudo foi pedir à professora que não diferenciase o ensino entre as duas condições. A intenção era tentar controlar a pedagogia para poder atribuir as diferenças nos resultados à modalidade da ferramenta introdutória. No entanto, ao fazer isso, perde-se uma oportunidade que poderia explicar a falta de transferência entre as ferramentas introdutórias e o Java. Na sala de aula com programação em blocos, depois que os alunos passaram para o Java, o ambiente de blocos não foi mais utilizado, nem foi mencionado como forma de traçar paralelos entre o Java e as ferramentas introdutórias usadas na primeira parte do curso. Ao adotar essa abordagem, limitamos a capacidade da professora de usar todos os recursos educacionais à sua disposição. Em entrevistas posteriores com a professora, ela discutiu como este estudo a ajudou a perceber o potencial da programação em blocos em cursos introdutórios de programação e comentou que, no futuro, continuaria a usar o ambiente de blocos, mas intercalando-o ao longo do ano. Ela disse que, no futuro, planejava introduzir cada novo conceito com a ferramenta baseada em blocos antes de passar para o Java, traçando paralelos diretos entre as duas. Esse padrão de introdução baseada em blocos, seguida por uma transição gradual baseada em texto, seria então repetido ao longo do ano. Embora não tenhamos dados para relatar os resultados dessa abordagem, o fato de a professora deste estudo ter decidido que essa seria a melhor estratégia para suas aulas demonstra o papel potencial da pedagogia em facilitar a transição entre ferramentas de programação. Da mesma forma, isso também sugere uma possível linha de pesquisa futura que esperamos explorar.

5.2. Implicações para o projeto de ambientes introdutórios de programação

Assim como a pedagogia é uma forma potencial de apoiar os alunos na transição da programação em blocos para a programação baseada em texto, o próprio ambiente de programação também pode facilitar esse processo. O ambiente de programação utilizado neste estudo (Pencil.cc) permitia que os alunos programassem em blocos ou em texto, mas não permitia a alternância entre os dois. O Pencil Code, ambiente no qual o Pencil.cc é baseado, foi especificamente projetado para suportar essa bidirecionalidade, permitindo que os alunos alternem livremente entre a programação em blocos e a programação baseada em texto. Pesquisas que analisam alunos trabalhando nesses tipos de ambientes de dupla modalidade mostram como dar esse controle ao usuário pode apoiar alunos com diferentes níveis de confiança, além de fornecer recursos para ajudá-los a criar programas bem-sucedidos (Matsuzawa et al., 2015; Weintrop & Holbert, 2017). Uma abordagem alternativa aos ambientes de dupla modalidade que suportam tanto blocos quanto texto são os ambientes híbridos, que combinam recursos de ferramentas baseadas em blocos e em texto em uma única interface. Por exemplo, editores baseados em frames são uma tentativa de viabilizar a edição em estilo de blocos, mas permitindo que os usuários utilizem principalmente o teclado, em vez da interação de arrastar e soltar da maioria das ferramentas baseadas em blocos (Kölling et al., 2017). Assim como ocorre com ferramentas de dupla modalidade, um crescente corpo de pesquisas demonstra que tais ambientes são uma abordagem promissora para auxiliar iniciantes na programação (Price et al., 2016; Weintrop & Wilensky, 2017c). O trabalho apresentado neste artigo sugere a necessidade de tais soluções de design para facilitar a transição de ferramentas baseadas em blocos para ferramentas baseadas em texto, mostrando que os ganhos obtidos em ambientes baseados em blocos não resultam em uma vantagem inicial em ferramentas baseadas em texto.

5.3. Escolhendo um ambiente de programação introdutório

Dado o crescente interesse na educação em computação, educadores e pais se deparam com a tarefa de escolher um ambiente de programação introdutório que sirva como primeiro contato do aluno com o mundo da programação. Ao escolher um ambiente de programação (ou modalidade de ambiente) para ser usado em um contexto educacional, é importante considerar a seguinte questão: qual é o objetivo dessa experiência introdutória em computação? Se a resposta for: preparar o aluno para o futuro ensino de ciência da computação, partindo do pressuposto de que esse caminho exigirá o aprendizado de programação em ferramentas profissionais baseadas em texto, então as descobertas deste artigo podem levar o educador a reconsiderar se uma ferramenta baseada em blocos é a melhor abordagem. Embora a estratégia introdutória baseada em blocos não tenha impedido o progresso do aluno ao passar para uma linguagem profissional, ela também não o facilitou de uma maneira diferente de uma ferramenta introdutória baseada em texto. Assim, se o educador tiver preocupações relacionadas à programação baseada em blocos (por exemplo, a logística de mudança de ambientes de aprendizagem ou preocupações dos alunos com a autenticidade das ferramentas baseadas em blocos) ou vir utilidade pedagógica em fazer com que os alunos digitem programas desde o primeiro dia, pode ser melhor começar com uma linguagem baseada em texto em vez de uma baseada em blocos.

Contudo, se o objetivo da experiência de aprendizagem for menos voltado para futuros cursos de ciência da computação e mais focado em objetivos mais amplos relacionados à alfabetização computacional e à preparação dos alunos para serem cidadãos informados em um mundo tecnológico, então uma ferramenta baseada em blocos pode ser um ambiente introdutório particularmente eficaz. Em outras palavras, definir qual é o objetivo final da experiência introdutória de aprendizagem em computação deve orientar a escolha da modalidade. Da mesma forma, essa mesma lógica se aplica à integração da programação em outras disciplinas ou contextos. Por exemplo, se a programação estiver sendo incluída em um curso de ciências para ajudar a desenvolver o raciocínio mecanicista de um fenômeno científico específico, então uma interface baseada em blocos pode ser apropriada. Alternativamente, se o objetivo for preparar um aluno para conduzir pesquisas científicas usando bibliotecas de programação científica existentes escritas em uma linguagem baseada em texto, então uma linguagem baseada em texto pode ser preferível. Coletivamente, este trabalho demonstra a importância de considerar os objetivos e os resultados desejados das atividades introdutórias de computação ao escolher a modalidade de programação.

5.4. Implicações além dos educadores e pesquisadores da ciência da computação

Embora o público-alvo principal deste trabalho sejam aqueles diretamente envolvidos na iniciativa de ajudar jovens aprendizes a terem experiências positivas e eficazes de aprendizagem em ciência da computação, há lições potenciais para um público mais amplo interessado no papel dos computadores na educação. A principal contribuição deste trabalho para um público mais amplo é destacar a existência de um "ponto cego do especialista" para aqueles que projetam tecnologias educacionais. A noção de um "ponto cego do especialista" é frequentemente discutida em relação a professores cujo conhecimento avançado de um assunto os impede de "enxergar" as dificuldades que os alunos possam ter com um determinado conceito (Nathan & Petrosino, 2003). No caso deste trabalho, as pessoas que projetam e implementam esses ambientes introdutórios de programação podem ver as conexões entre uma instrução condicional baseada em blocos e seu equivalente textual, e podem presumir que os alunos também verão a conexão. Esta pesquisa mostra que essa conexão não é tão clara para o aluno quanto os projetistas poderiam ter esperado. A implicação disso, além do design de ambientes introdutórios de programação, é que todos os designers e pesquisadores de tecnologia educacional devem avaliar criticamente qual conhecimento pode ser considerado óbvio em seus projetos e destacar o papel dos estudos empíricos para explorar possíveis pontos cegos dos especialistas.

Um segundo ponto de interesse relacionado a este trabalho, para aqueles fora da comunidade de ensino de ciência da computação, decorre do desenho do estudo e do poder explicativo que ele proporciona. Dado o nosso interesse na prática em sala de aula e o desejo de obter resultados relevantes tanto para pesquisadores quanto para educadores, o estudo foi especificamente concebido para ser útil a ambos os públicos. Para tanto, buscou-se tornar o ambiente de estudo o mais autêntico possível, ao mesmo tempo em que se faziam as adaptações necessárias para permitir comparações diretas entre as condições experimentais. Isso significou inserir o estudo em turmas existentes e trabalhar com um professor experiente. Embora isso tenha tornado o estudo mais desafiador, principalmente em termos de encontrar um ambiente adequado e obter as aprovações institucionais necessárias, o resultado foi um conjunto de dados que resiste ao escrutínio tanto de acadêmicos quanto de professores. Se o objetivo de um projeto de pesquisa é moldar a prática em sala de aula, estar presente nas salas de aula é um passo importante para alcançar esse objetivo.

Uma contribuição final deste trabalho é demonstrar uma análise multifacetada dos potenciais papéis da tecnologia na educação. A seção de resultados apresentou descobertas relacionadas à aprendizagem de conteúdo, aos impactos atitudinais e às práticas associadas ao domínio do conteúdo. O estudo foi concebido para dissociar essas diferentes dimensões da aprendizagem, o que significa que foi possível observar diferenças significativas em qualquer dimensão, independentemente dos resultados das outras duas. Dessa forma, houve múltiplas oportunidades de encontrar diferenças nos resultados entre as duas condições. Embora nenhuma diferença tenha surgido, este estudo demonstra uma possível maneira de analisar os resultados educacionais quando a aprendizagem é definida de forma mais abrangente do que apenas um aumento de conhecimento.

5.5. Limitações e trabalhos futuros

Este estudo foi concebido para responder a questões fundamentais de pesquisa relacionadas a se e como a modalidade introdutória de programação (baseada em blocos ou em texto) impacta os alunos em sua transição para linguagens de programação profissionais baseadas em texto. Embora este estudo forneça informações sobre essa questão, existem limitações nos resultados. O primeiro conjunto de limitações deste estudo está relacionado às características excepcionais dos participantes. O estudo foi realizado em uma escola de admissão seletiva e em turmas de ciência da computação onde os alunos precisavam se inscrever para participar. Isso significa que o estudo foi composto por alunos que historicamente obtiveram sucesso em contextos educacionais formais e demonstraram aptidão para ciência da computação. Assim, as conclusões não se aplicam necessariamente a todos os alunos, especialmente àqueles sem interesse em ciência da computação. A proporção de alunos do sexo masculino para o feminino é outra limitação do estudo, visto que os homens estavam sobrerrepresentados. Por fim, o professor que participou deste estudo é um educador de ciência da computação excepcional e experiente. Não está claro como os resultados deste estudo seriam diferentes em uma sala de aula liderada por um professor menos experiente ou menos confiante.

instrutor. Embora nenhuma dessas limitações comprometa as conclusões, elas restringem a generalização deste estudo e apontam caminhos para trabalhos futuros. Uma direção clara seria a realização de um estudo de replicação em uma escola diferente, com um grupo diferente de alunos. Existem desafios associados à condução de tal estudo e ao recrutamento de professores e alunos, mas seria um passo importante para ampliar a generalização deste trabalho.

Uma segunda limitação do estudo está relacionada à sua duração de 15 semanas. As dez semanas do estudo foram tempo suficiente para introduzir os conceitos, mas representam apenas uma fração do curso anual. É possível que diferenças entre as condições surjam mais tarde no semestre ou que diferentes modalidades impactem a aprendizagem de alguns conceitos que não foram abordados nas primeiras 10 semanas do curso.

Embora o estudo não tenha apresentado motivos para acreditar que seja esse o caso, é possível que sejam necessárias mais de 10 semanas para que as diferenças se manifestem. Uma futura edição deste estudo, com duração de um ano inteiro, poderia explorar essa possibilidade mais a fundo.

Este estudo também apresenta limitações potenciais decorrentes dos ambientes de programação, linguagens e currículos específicos utilizados.

Embora o Pencil.cc seja um ambiente razoavelmente representativo tanto para a categoria mais ampla de ambientes de programação baseados em blocos quanto para aqueles baseados em texto, em ambos os casos ele carece de alguns recursos comuns. Por exemplo, o modo baseado em blocos possui um vocabulário e uma estrutura de instruções relativamente restritos em comparação com ferramentas como o Scratch. Ao mesmo tempo, o editor de texto inclui recursos comuns, como realce de sintaxe, mas não inclui funcionalidades comuns como autocompletar ou sugestões de código incorporadas. Além disso, a decisão de usar o CoffeeScript como linguagem de programação baseada em texto para a parte introdutória do estudo é apenas uma das muitas alternativas de linguagem possíveis. Assim como no caso das limitações relacionadas aos participantes, a realização de trabalhos futuros explorando a transição entre diferentes linguagens e diferentes implementações do paradigma baseado em blocos são caminhos promissores para pesquisas futuras.

Uma limitação relacionada decorre da transição de um currículo introdutório, no qual os alunos escreviam programas procedurais, para um currículo Java orientado a objetos. Embora essa seja uma mudança conceitual, acreditamos que essa transição teve um impacto relativamente pequeno nas descobertas apresentadas a seguir, por alguns motivos. Primeiro, embora os programas escritos na parte Java do estudo fossem orientados a objetos, o conteúdo intensivo baseado em objetos raramente foi encontrado. Os programas desenvolvidos focavam mais em entrada/saída básica, criação de variáveis (frequentemente usando tipos primitivos) e chamada de funções. Quando objetos eram encontrados, os alunos geralmente recebiam modelos de programa para seguir, que incluíam o código para instanciação de objetos. Um segundo motivo para acreditar que o impacto foi relativamente limitado reside na forma como o Pencil.cc lida com sprites. Quando um novo sprite é criado, ele é tratado como um objeto. O comando para criar um novo sprite é: `s = new Sprite()`, e as instruções para o novo sprite utilizam a notação de ponto (`egsfd.100`). Assim, os alunos encontraram conceitos e sintaxe básicos de orientação a objetos durante a parte introdutória do curso. Por fim, como todos os alunos fizeram a mesma transição, esperamos que qualquer impacto decorrente dessa transição seja sentido por ambos os grupos. Dito isso, é possível que essa mudança de paradigma tenha tido algum impacto nos resultados de todos os alunos, e isso é considerado uma limitação.

Outra limitação relaciona-se às escolhas metodológicas feitas sobre como operacionalizamos o "impacto" neste estudo. Ao focarmos na compreensão conceitual, nos resultados atitudinais e nas práticas de programação, destacamos apenas algumas das muitas facetas da aprendizagem de programação. Ausentes da análise apresentada estavam medidas como a correção do programa, a qualidade do programa ou a capacidade geral de resolver problemas reais por meio da programação. Embora essas medidas sejam desejáveis, elas não se adequavam bem ao contexto em que este trabalho foi realizado. Primeiro, em relação à correção do programa, a simplicidade dos programas criados e o nível de suporte oferecido em sala de aula contribuíram para isso. O professor incentivava os alunos a se ajudarem mutuamente e frequentemente trabalhava com eles durante as aulas, especialmente com aqueles que apresentavam dificuldades. Como resultado, os programas finais entregues como tarefas eram, em sua grande maioria, corretos para a tarefa, tornando a correção do programa uma medida pouco útil neste contexto específico. Em vez da correção do programa, utilizamos características do processo de criação do programa como forma de obter insights sobre a compreensão emergente dos conceitos pelos alunos e o desenvolvimento de suas habilidades de programação. Isso nos permite observar o progresso do aluno ao longo do tempo, em vez de nos basearmos apenas no programa final. Avaliar a qualidade do programa é difícil por razões semelhantes, decorrentes da simplicidade das tarefas e da natureza complexa e desorganizada da pesquisa em sala de aula. Em termos de tentar avaliar a capacidade dos alunos de resolver problemas do mundo real usando programação, que é o objetivo final da disciplina, após 10 semanas de aprendizado de Java neste curso, as tarefas atribuídas aos alunos não estavam nesse nível de sofisticação. Ao final do estudo, os alunos ainda estavam criando programas relativamente simples, focados na introdução e aplicação de conceitos básicos, em vez de empregar esses conceitos para resolver problemas reais. Mais tarde no ano letivo, os programas passam a abordar desafios mais aplicados e autênticos, mas isso ocorre bem depois do trabalho inicial, que foi o foco das primeiras 10 semanas.

Além dessa lista de limitações, existem também outras direções potenciais para trabalhos futuros que esperamos explorar. Uma possibilidade que foi considerada, mas não seguida neste estudo, é investigar como a modalidade afeta diferencialmente diferentes tipos de alunos. Por exemplo, alunos com dificuldades em programação ou que inicialmente demonstram menos interesse pela disciplina percebem mais, menos ou diferentes benefícios de uma modalidade de programação em comparação com outra? Outra direção para trabalhos futuros é adotar uma abordagem semelhante, centrada na modalidade, para conceitos além da programação e da ciência da computação. Como argumentado por Wilensky & Papert (2010), há grande potencial e muito trabalho a ser feito na reimaginação da infraestrutura representacional de um domínio e no papel potencial de sistemas representacionais mediados computacionalmente. À medida que novos campos se tornam cada vez mais computacionais, novas ferramentas, interfaces e modalidades estão sendo inventadas para dar suporte a novos empreendimentos computacionais. Projetos de pesquisa semelhantes a este trabalho poderiam se encaixar bem entre as diversas atividades que acompanham a formação de novas ferramentas e práticas tecnologicamente aprimoradas e o surgimento mais amplo de novos campos computacionais.

6. Conclusões

Embora as linguagens baseadas em blocos tenham explodido em popularidade, relativamente poucas pesquisas foram feitas para demonstrar que os alunos que aprendem nesses ambientes estão efetivamente fazendo a transição de conhecimentos e práticas emergentes para linguagens textuais mais tradicionais, como Java (Blikstein, 2018). O objetivo deste trabalho é explorar essa transição em um ambiente de sala de aula do ensino médio para entender se e

Como a modalidade facilita essa transição? Versões dessa questão foram respondidas de diversas maneiras na literatura (Armoni et al., 2015; Lewis, 2010; Price & Barnes, 2015; Saito et al., 2016; Weintrop, 2016), mas nunca em um contexto quase-experimental onde fatores como professor, currículo e tempo dedicado à tarefa foram controlados e os alunos pertenciam ao mesmo corpo discente. Os resultados deste estudo em sala de aula do ensino médio mostram que os alunos obtiveram maiores ganhos de aprendizagem em ambientes baseados em blocos em comparação com alternativas isomórficas baseadas em texto, mas que esses ganhos não resultaram em diferença no ensino subsequente baseado em texto no que diz respeito à aprendizagem conceitual, mudanças de atitude ou práticas de programação bem-sucedidas.

Ao demonstrar que os ganhos conceituais obtidos em ferramentas introdutórias baseadas em blocos não se transferem automaticamente para uma linguagem profissional baseada em texto, destacamos a necessidade de educadores e desenvolvedores de ferramentas facilitarem essa transição. Além disso, ao identificar como a modalidade molda as atitudes dos alunos em relação à área da ciência da computação e como essa atitude se altera à medida que eles mudam de linguagem de programação, podemos auxiliar educadores a tomarem decisões informadas sobre a melhor forma de acolher os alunos no mundo da computação. À medida que o papel da computação e da tecnologia continua a crescer na sociedade, preparar os jovens para serem participantes informados nesse cenário tecnológico torna-se fundamental. Identificar os pontos fortes e as limitações das ferramentas introdutórias de programação que desempenham um papel na construção dessa alfabetização computacional básica é um componente importante desse processo. O objetivo deste trabalho é nos aproximar da compreensão de como melhor preparar todos os alunos para o futuro computacional que os aguarda.

Referências

- Altadmri, A., & Brown, NCC (2015). 37 milhões de compilações: Investigando erros de programação de iniciantes em dados de estudantes em larga escala. Anais do 46º Simpósio Técnico da ACM sobre Educação em Ciência da Computação - SIGCSE '15. Apresentado no 46º Simpósio Técnico da ACM (pp. 522–527). Kansas City, Missouri, EUA: ACM Press. <https://doi.org/10.1145/2676723.2677258>.
- Associação Americana de Mulheres Universitárias (1994). Prejudicando as meninas, prejudicando a América. Washington, DC: Fundação Educacional AAUW.
- Armoni, M., Meierbaum-Salant, O., & Ben-Ari, M. (2015). Do zero à programação "real". ACM Trans. Comput. Educ. TOCE, 14(25), 1–15.
- Bart, AC, Tibau, J., Kafura, D., Shaffer, CA, & Tilevich, E. (2017). Projeto e avaliação de um ambiente baseado em blocos com um contexto de ciência de dados. IEEE Trans. Emerg. Top. Comput. 1–1 <https://doi.org/10.1109/TETC.2017.2729585>.
- Bau, D. (2015). Droplet, um editor baseado em blocos para código de texto. J. Comput. Sci. Coll. 30, 138–144.
- Bau, D., Bau, DA, Dawson, M., & Pickens, CS (2015). Pencil code: Block code for a text world. Proceedings of the 14th International Conference on interaction design and Children (pp. 445–448). New York, NY, USA: IDC '15. ACM. <https://doi.org/10.1145/2771839.2771875>.
- Bau, D., Gray, J., Kelleher, C., Sheldon, J., & Turbap, F. (2017). Programação aprendível: Blocos e além. Communications of the ACM, 60, 72–80. <https://doi.org/10.1145/3015455>.
- Berland, M., Martin, T., Benton, T., Petrick Smith, C., & Davis, D. (2013). Utilizando análises de aprendizagem para compreender as trajetórias de aprendizagem de programadores iniciantes. O Revista de Ciências da Aprendizagem, 22, 564–599. <https://doi.org/10.1080/10508406.2013.836655>.
- Bishop-Clark, C., Courte, J., & Howard, EV (2006). Programação em pares com Alice para melhorar a confiança, o prazer e o desempenho. Journal of Educational Computing Research, 34, 213–228. <https://doi.org/10.2190/CFKF-UJGC-JG1Q-7T40>.
- Blanchard, J. (2017). Ambientes híbridos: uma ponte entre blocos e texto. Anais da Conferência Internacional de Pesquisa em Educação Computacional da ACM de 2017, ICER '17. (págs. 295–296). Nova York, NY, EUA: ACM. <https://doi.org/10.1145/3105726.3105743>.
- Blikstein, P. (2018). Educação em ciência da computação pré-universitária: uma pesquisa da área. Mountain View, CA: Google LLC.
- Blikstein, P., Worsley, M., Piech, C., Sahami, M., Cooper, S., & Koller, D. (2014). Pluralismo em programação: usando análise de aprendizagem para detectar padrões na aprendizagem de Programação de computadores. The Journal of the Learning Sciences, 23, 561–599. <https://doi.org/10.1080/10508406.2014.954750>.
- Brown, NCC, Mönig, J., Bau, A., & Weintrop, D. (2016). Direções Futuras da Programação Baseada em Blocos. Anais do 47º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (pp. 315–316). Nova York, NY, EUA: SIGCSE'16. ACM. <https://doi.org/10.1145/2839509.2844661>.
- Bruckman, A., Biggers, M., Ericson, B., McKlin, T., Dimond, J., DiSalvo, B., et al. (2009). Georgia computes!: Melhorando o caminho para o ensino de computação. ACM SIGCSE Boletim. ACM.
- Caspersen, M. (2018). Ensinar programação. Em S. Sentance, E. Barendsen e C. Schulte (Eds.). Educação em ciência da computação: Perspectivas sobre ensino e aprendizagem (pp. 109–130). Bloomsbury Publishing.
- Cilburn, DC (2008). Opiniões dos alunos sobre Alice em CS1. Conferência Frontiers in Education, 2008. FIE 2008. 38ª Conferência Anual IEEE T3B–1.
- Code.org (2017). A 5ª Hora do Código chegou! Code.org. 20/09/2018 <https://medium.com/@codeorg/the-5th-hour-of-code-is-here-5b9ed3c29c50>.
- Curriculos do Code.org (2019). Code.org.
- Cohen, L., Manion, L., & Morrison, K. (2007). Métodos de pesquisa em educação (6ª ed.). Nova York: Routledge, Londres. .
- Cooper, S., Dann, W., & Pausch, R. (2000). Alice: Uma ferramenta 3D para conceitos introdutórios de programação. J. Comput. Sci. Coll. 15, 107–116.
- Cuny, J. (2015). Transformando o ensino de computação do jardim de infância ao ensino médio: uma atualização e um apelo à ação. ACM Inroads, 6, 54–57. <https://doi.org/10.1145/2809795>.
- Danielak, BA (2014). Como os estudantes de engenharia elétrica projetam programas de computador. College Park, MD: Universidade de Maryland.
- Dann, W., Cosgrove, D., Slater, D., Culyba, D., & Cooper, S. (2012). Transferência mediada: Alice 3 para Java. Anais do 43º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (pp. 141–146). ACM.
- Dorn, B., & Elliott Tew, A. (2015). Validação empírica e aplicação da pesquisa de atitudes em computação. Educação em Ciência da Computação, 25, 1–36. <https://doi.org/10.1080/08993408.2015.1014142>.
- Duncan, C., Bell, T., & Tanimoto, S. (2014). Seu filho de 8 anos deveria aprender a programar? Anais do 9º Workshop em educação em computação no ensino fundamental e médio (pp. 60–69). Nova York, NY, EUA: WIPSCSE '14. ACM. <https://doi.org/10.1145/2670757.2670774>.
- Fay, MP, & Proschan, MA (2010). Wilcoxon-Mann-Whitney ou teste t? Sobre pressupostos para testes de hipóteses e múltiplas interpretações de regras de decisão. Estatística Pesquisas, 4, 1–39. <https://doi.org/10.1214/09-SS051>.
- Flowers, T., Carver, CA, & Jackson, J. (2004). Capacitando alunos e construindo confiança em programadores iniciantes por meio do Gauntlet. Frontiers in education, 2004. FIE 2004. 34ª Anual. Apresentado na conferência Frontiers in Education, 2004. FIE 2004. 34ª Anual: Vol. 1, (pp. T3H10–T3H13). . <https://doi.org/10.1109/FIE.2004.1408551>.
- Franklin, D., Skifstad, G., Rolock, R., Mehrotra, I., Ding, V., Hansen, A., Weintrop, D., & Harlow, D. (2017). Utilizando o desempenho de alunos do Ensino Fundamental II para Compreendendo a Sequenciação Conceitual em um Currículo Baseado em Blocos. Anais do Simpósio Técnico ACM SIGCSE de 2017 sobre Educação em Ciência da Computação (pp. 231–236). Nova York, NY, EUA: SIGCSE '17. ACM. <https://doi.org/10.1145/3017680.3017760>.
- Fraser, N. (2015). Dez coisas que aprendemos com o Blockly. Workshop IEEE Blocks and Beyond de 2015 (Blocks and Beyond). Apresentado no Workshop IEEE Blocks and Beyond de 2015. Workshop (blocos e além) (pp. 49–50). . <https://doi.org/10.1109/BLOCKS.2015.7369000>.
- Garcia, D., Harvey, B., & Barnes, T. (2015). A beleza e a alegria da computação. ACM Inroads, 6, 71–79. <https://doi.org/10.1145/2835184>.
- Garlick, R., & Cankaya, EC (2010). Usando Alice em CS1: um experimento quantitativo. Anais da Décima Quinta Conferência Anual sobre Inovação e Tecnologia no Ensino de Ciência da Computação (pp. 165–168). ACM.
- Good, J. (2018). Ambientes de programação para iniciantes: Reduzindo as barreiras, apoiando o progresso. Métodos inovadores, ferramentas amigáveis ao usuário, codificação e design. Abordagens em programação orientada para pessoas (pp. 1–41). IGI Global.
- Goode, J., Chapman, G., & Margolis, J. (2012). Além do currículo: O programa de exploração da ciência da computação. ACM Inroads, 3, 47–53.
- Good, J., & Howland, K. (2017). Linguagem de programação, linguagem natural? Apoiando as diversas atividades computacionais de programadores iniciantes. Journal of Visual

- Languages & Computing, 39, 78–92. <https://doi.org/10.1016/j.jvlc.2016.10.008>.
- Grover, S., & Basu, S. (2017). Medindo a aprendizagem do aluno na programação introdutória baseada em blocos: Examinando concepções errôneas de loops, variáveis e lógica booleana. *Anais do Simpósio Técnico ACM SIGCSE de 2017 sobre educação em ciência da computação* (pp. 267–272). Nova York, NY: ACM Press. <https://doi.org/10.1145/3017680.3017723>.
- Grover, S., Pea, R., & Cooper, S. (2015). Projetando para uma aprendizagem mais profunda em um curso de ciência da computação semipresencial para alunos do ensino fundamental. *Educação em Ciência da Computação*, 25, 199–237. <https://doi.org/10.1080/08993408.2015.1033142>.
- Harvey, B. (1997). *Estilo de logotipo de ciência da computação: além da programação*. The MIT Press.
- Hill, C., Dwyer, H., Martinez, T., Harlow, D., & Franklin, D. (2015). Pisos e Flexibilidade: Projetando um ambiente de programação para salas de aula do 4º ao 6º ano. *Anais do 46º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (pp. 546–551). ACM.
- Homer, M., & Noble, J. (2017). Lições na combinação de programação baseada em blocos e textual. *J. Vis. Lang. Sentient Syst.* 3, 22–39. <https://doi.org/10.18293/VLSS2017>.
- Horstmann, CS (2012). *Conceitos de Java: Objetos iniciais* (7ª edição). Hoboken, NJ: Wiley.
- Howland, K., & Good, J. (2014). Aprendendo a comunicar computacionalmente com flip: Uma linguagem de programação bimodal para criação de jogos. *Computers & Education*. <https://doi.org/10.1016/j.compedu.2014.08.014>.
- Jackson, J., Cobb, MJ, & Carver, C. (2005). Identificando os principais erros de Java para programadores iniciantes. *Conferência Frontiers in Education*. IEEE.
- Jadud, MC (2005). Uma primeira olhada no comportamento de compilação de novatos usando BlueJ. *Educação em Ciência da Computação*, 15, 25–40.
- Jadud, MC, & Henriksen, P. (2009). Ferramentas flexíveis e reutilizáveis para o estudo de programadores iniciantes. *Anais do Quinto Workshop Internacional sobre Pesquisa em Educação em Computação* (pp. 37–42). ACM.
- Johnsgard, K., & McDonald, J. (2008). Utilizando Alice em cursos introdutórios para melhorar as taxas de sucesso em programação I. 21ª Conferência IEEE sobre Engenharia de Software. *educação e treinamento*, 2008 (pp. 129–136). CSEET '08. <https://doi.org/10.1109/CSEET.2008.35>.
- Kelleher, C., & Pausch, R. (2007). Usando storytelling para motivar a programação. *Communications of the ACM*, 50, 58–64.
- Kelleher, C., Pausch, R., & Kiesler, S. (2007). A história de Alice motiva meninas do ensino fundamental a aprender programação de computadores. *Anais da Conferência SIGCHI sobre Fatores humanos em sistemas computacionais* (pp. 1455–1464).
- Kölling, M., Brown, NCC, & Altadmiri, A. (2015). Edição baseada em frames: facilitando a transição de blocos para programação baseada em texto. *Anais do Workshop em Educação em informática no ensino fundamental e médio* (pp. 29–38). Nova York, NY, EUA: WiPSCE '15. ACM. <https://doi.org/10.1145/2818314.2818331>.
- Kölling, M., Brown, NCC, & Altadmiri, A. (2017). Edição baseada em quadros. *J. Vis. Lang. Sentient Syst.* 3, 40–67. <https://doi.org/10.18293/VLSS2017>.
- Kölling, M., & McKay, F. (2016). Avaliação heurística para sistemas de programação para iniciantes. *Transactions on Computing Education*, 16, 12. 1–12:30 <https://doi.org/10.1145/2872521>.
- Levenshtein, VI (1966). Códigos binários capazes de corrigir deleções, inserções e inversões. *Física Soviética* (pp. 707–710).
- Lewis, CM (2010). Como o ambiente de programação molda a percepção, a aprendizagem e os objetivos: Logo vs. Scratch. *Anais do 41º Simpósio Técnico da ACM sobre educação em ciência da computação*. Nova York, NY (pp. 346–350).
- Malan, DJ, & Leitner, HH (2007). Scratch para cientistas da computação iniciantes. *Boletim ACM SIGCSE*. ACM.
- Maloney, JH, Peppler, K., Kafai, Y., Resnick, M., & Rusk, N. (2008). Programação por escolha: jovens urbanos aprendendo programação com Scratch. *ACM SIGCSE Bull.* 40, 367–371.
- Maloney, JH, Resnick, M., Rusk, N., Silverman, B., & Eastmond, E. (2010). A linguagem e o ambiente de programação Scratch. *ACM Trans. Comput. Educ.* TOCE, 10, 16.
- Margolis, J., & Fisher, A. (2003). *Desbloqueando o clube: Mulheres na computação*. The MIT Press.
- Matsuzawa, Y., Ohata, T., Sugiyama, M., & Sakai, S. (2015). Migração de linguagem em programação introdutória não relacionada à ciência da computação por meio de ambiente de tradução mútua de linguagem. *Anais do 46º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (pp. 185–190). ACM Press. <https://doi.org/10.1145/2676723.2677230>.
- McDowell, C., Werner, L., Bullock, HE, & Fernald, J. (2006). A programação em pares melhora a retenção de alunos, a confiança e a qualidade do programa. *Comunicações da ACM*, 49, 90–95. <https://doi.org/10.1145/1145287.1145293>.
- Meerbaum-Salant, O., Armoni, M., & Ben-Ari, MM (2010). Aprendendo conceitos de ciência da computação com Scratch. *Anais do sexto Workshop Internacional sobre pesquisa em educação computacional* (pp. 69–76).
- Mönig, J., Ohshima, Y., & Maloney, J. (2015). Blocos ao seu alcance: Desfocando a linha divisória entre blocos e texto em GP. *Workshop IEEE Blocks and Beyond de 2015 (blocos e além)*. Apresentado no Workshop IEEE Blocks and Beyond de 2015 (blocos e além) (pp. 51–53). <https://doi.org/10.1109/BLOCKS.2015.7369001>.
- Nathan, MJ, & Petrosino, A. (2003). Ponto cego do especialista entre professores em formação. *American Educational Research Journal*, 40, 905–928. <https://doi.org/10.3102/00028312040004905>.
- Noone, M., & Mooney, A. (2018). Linguagens de programação visual e textual: uma revisão sistemática da literatura. *Journal of Computers in Education*. 5, 149–174. <https://doi.org/10.1007/s40692-018-0101-5>.
- Papert, S. (1980). *Mindstorms: Crianças, computadores e ideias poderosas*. Nova York: Basic books.
- Piech, C., Sahami, M., Koller, D., Cooper, S., & Blikstein, P. (2012). Modelagem de como os alunos aprendem a programar. *Anais do 43º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (pp. 153–160). ACM.
- Powers, K., Ecott, S., & Hirshfeld, LM (2007). Através do espelho: Ensinando CS0 com Alice. *ACM SIGCSE Bull.* 39, 213–217.
- Price, TW, & Barnes, T. (2015). Comparando interfaces textuais e em bloco em um ambiente de programação para iniciantes. Apresentado no ICER '15. *ACM Press* <https://doi.org/10.1145/2787622.2787712>.
- Price, TW, Brown, NC, Lipovac, D., Barnes, T., & Kölling, M. (2016). Avaliação de um editor de programação baseado em frames. *Anais da Conferência ACM de 2016 sobre Pesquisa internacional em educação computacional* (pp. 33–42). ACM.
- Resnick, M., Silverman, B., Kafai, Y., Maloney, J., Monroy-Hernández, A., Rusk, N., et al. (2009). Scratch: Programação para todos. *Comunicações da ACM*, 52, 60.
- Rodríguez Corral, JM, Ruiz-Rube, I., Civit Balcells, A., Mota-Macias, JM, Morgado-Estevéz, A., & Dodero, JM (2019). Um estudo sobre a adequação das linguagens visuais para programadores de robôs não especialistas. *IEEE Access*, 7, 17535–17550. <https://doi.org/10.1109/ACCESS.2019.2895913>.
- Ruf, A., Mühling, A., & Hubwieser, P. (2014). Scratch vs. Karel: Impacto nos resultados de aprendizagem e na motivação. *ACM Press* <https://doi.org/10.1145/2670757.2670772>.
- Saito, D., Washizaki, H., & Fukazawa, Y. (2016). Análise dos efeitos de aprendizagem entre ambientes de programação para iniciantes baseados em texto e em recursos visuais. Apresentado na 8ª Conferência Internacional de Educação em Engenharia (ICEED) do IEEE de 2016 (pp. 208–213). <https://doi.org/10.1109/ICEED.2016.7856073>.
- Scholtz, J., & Wiedenbeck, S. (1990). Aprendendo uma segunda e subsequentes linguagens de programação: um problema de transferência. *International Journal of Human Computer Interaction*, 2, 51–72.
- Shapiro, RB, & Ahrens, M. (2016). Além dos blocos: Sintaxe e semântica. *Communications of the ACM*, 59, 39–41. <https://doi.org/10.1145/2903751>.
- Simon e Snowdon, S. (2014). Questões de múltipla escolha versus questões de texto livre explicando códigos em exames. Apresentado nos Anais da 14ª Conferência Internacional de Chamada Koli. Conferência sobre pesquisa em educação computacional (pp. 91–97). ACM. <https://doi.org/10.1145/2674683.2674701>.
- Streiner, DL (2003). Começando pelo começo: Uma introdução ao coeficiente alfa e à consistência interna. *Journal of Personality Assessment*, 80, 99–103. https://doi.org/10.1207/S15327752JPA8001_18.
- Tabet, N., Gedawy, H., Alshikhabobakr, H., & Razak, S. (2016). De Alice ao Python. Introduzindo programação baseada em texto no ensino fundamental. *Anais da Conferência ACM de 2016 sobre Inovação e Tecnologia no Ensino de Ciência da Computação - ITICSE '16*. Apresentado na Conferência ACM de 2016 (pp. 124–129). Arequipa, Peru: ACM Press. <https://doi.org/10.1145/2899415.2899462>.
- Tangney, B., Oldham, E., Conneely, C., Barrett, S., & Lawlor, J. (2010). Pedagogia e processos para uma oficina de extensão em programação de computadores — a ponte para a universidade. *modelo. Educ. IEEE Trans. On*, 53, 53–60.
- Tavani, CM, & Losh, SC (2003). Motivação, autoconfiança e expectativas como preditores do desempenho acadêmico entre nossos alunos do ensino médio [WWW Documento]. *Child Study J.* 7.22.19 <http://link.galegroup.com/apps/doc/A116924600/HRC?aid=googlescholar>.
- Tempel, M. (2013). Programação em blocos. Vol. 9. CSTA Voice.
- Tew, AE, Dorn, B., & Schneider, O. (2012). Rumo a uma pesquisa validada sobre atitudes em relação à computação. *Anais da Nona Conferência Internacional Anual sobre Internacional*

- pesquisa em educação computacional (pp. 135–142). ACM.
- Vihavainen, A., Luukkainen, M., & Ihtola, P. (2014). Análise dos níveis de granularidade do snapshot do código-fonte. ACM Press <https://doi.org/10.1145/2656450.2656473>.
- Weintrop, D. (2016). A modalidade importa: Compreendendo os efeitos da representação da linguagem de programação em salas de aula de ciência da computação do ensino médio (Dissertação de doutorado) Evanston, IL: Universidade Northwestern.
- Weintrop, D., & Holbert, N. (2017). De blocos a texto e vice-versa: padrões de programação em um ambiente de dupla modalidade. Anais da Conferência Técnica ACM SIGCSE de 2017. Simpósio sobre educação em ciência da computação (pp. 633–638). Nova York, NY, EUA: SIGCSE '17. ACM. <https://doi.org/10.1145/3017680.3017707>.
- Weintrop, D., & Wilensky, U. (2018). Como as modalidades baseadas em blocos, baseadas em texto e híbridas bloco/texto moldam as práticas de programação de iniciantes. Int. J. Child Comput. Interact. 17, 83–92. <https://doi.org/10.1016/j.ijcci.2018.04.005>.
- Weintrop, D., & Wilensky, U. (2017a). Comparando a programação baseada em blocos e a programação baseada em texto em salas de aula de ciência da computação do ensino médio. ACM Trans. Comput. Educ. TOCE, 18, 3. <https://doi.org/10.1145/3089799>.
- Weintrop, D., & Wilensky, U. (2017b). Como as linguagens baseadas em blocos auxiliam os novatos: Uma estrutura para categorizar as affordances baseadas em blocos. J. Vis. Lang. Sentient Syst. 3, 92–100. <https://doi.org/10.18293/VLSS2017-006>.
- Weintrop, D., & Wilensky, U. (2017c). Entre um bloco e uma fonte: projetando e avaliando ambientes de programação híbridos. Anais da conferência de 2017. sobre design de interação e crianças (pp. 183–192). Nova York, NY, EUA: IDC '17. ACM. <https://doi.org/10.1145/3078072.3079715>.
- Weintrop, D., & Wilensky, U. (2015a). Utilizando avaliações comutativas para comparar a compreensão conceitual em programas baseados em blocos e em texto. Anais da décima primeira conferência internacional anual sobre pesquisa em educação computacional (pp. 101–110). Nova York, NY, EUA: ICER '15. ACM. <https://doi.org/10.1145/2787622.2787721>.
- Weintrop, D., & Wilensky, U. (2015b). Bloquear ou não bloquear, eis a questão: percepções dos alunos sobre programação baseada em blocos. Anais da 14ª Conferência Internacional sobre Design de Interação e Crianças (pp. 199–208). Nova York, NY, EUA: IDC '15. ACM. <https://doi.org/10.1145/2771839.2771860>.
- Weintrop, D. (2019). Programação baseada em blocos no ensino de ciência da computação. Commun. ACM, 62(8), 22–25. <https://doi.org/10.1145/3341221>.
- Wiedenbeck, S. (1993). Uma análise de programadores iniciantes aprendendo uma segunda linguagem. Estudos empíricos de programadores: Quinto Workshop: Artigos apresentados no Quinto Workshop sobre estudos empíricos de programadores, 3 a 5 de dezembro de 1993, Palo Alto, CA (p. 187). Intellect Books.
- Wilensky, U., & Papert, S. (2010). Reestruturas: Reformulando as disciplinas do conhecimento por meio de novas formas representacionais. Em J. Clayson, & I. Kallas (Eds.). Atas da Conferência de Construcionismo de 2010. Paris, França.
- Wilson, A., & Moffat, DC (2010). Avaliando o Scratch para introduzir crianças mais novas à programação. In: Anais do 22º Grupo Anual de Interesse em Programação Psicológica da Universidade Carlos III Madr. Leganés, Espanha.